Bionetta: Efficient Client-Side zkML with Constant Verification Complexity

***Abstract*—The widespread adoption of neural networks necessitates robust mechanisms to ensure the privacy and integrity of interactions with machine learning models. A prominent approach is to use succinct proving systems to ensure the correctness of neural network inference, optionally concealing model weights or user inputs. Such zero-knowledge machine learning (zkML) solutions enable a broad set of applications, from ensuring the correctness of Large Language Model output to building privacy-preserving biometric authentication systems. However, implementing zkML in practice is challenging: encoding model inference as arithmetic circuits produces large constraint systems, causing substantial overhead in proving, verification, and proof sizes. Moreover, many decentralized identity applications processing biometric data impose two demanding requirements: (1) fast proving on lightweight client devices and (2) succinct and cheap verification on-chain.**

This paper presents Bionetta, an R1CS-based zkML framework that addresses both challenges. Its design includes a weight-embedding technique that compiles model parameters directly into the R1CS instance, thereby eliminating constraints for all linear operations in the neural network; a zero-knowledge-efficient arithmetization of floating-point operations; and *UltraGroth*, a modification of Groth16 protocol with lookup table support that reduces circuit sizes by a logarithmic factor. While compatible with any R1CS proving system, Bionetta paired with UltraGroth achieves 320-byte constant-sized proofs (up to $50\times$ improvement over existing schemes), verification in four pairings (up to $700\times$ improvement), and a verification key size below 10 KB, requiring only a small per-circuit trusted setup (up to $40\times$ improvement). We further demonstrate that Bionetta enables efficient client-side proof generation: a full MobileNetV2 inference proof can be generated in under 25 seconds on a standard mobile device.

***Index Terms*—Zero-Knowledge Proofs, Machine Learning, Client-Side Proving, Lookup Arguments**

1. Introduction

Zero-knowledge proof systems have been extensively used in various applications [67], ranging from blockchain scalability solutions [47], [78] and zero-knowledge virtual machines (zkVMs) [24], [65] to privacy-preserving identity protocols such as *Rarimo* [73]. In particular, such systems utilize the special case of zero-knowledge proofs called zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) [63], allowing reducing both the verification time and proof size to $O(\text{polylog}(N))$ with N being the circuit size.

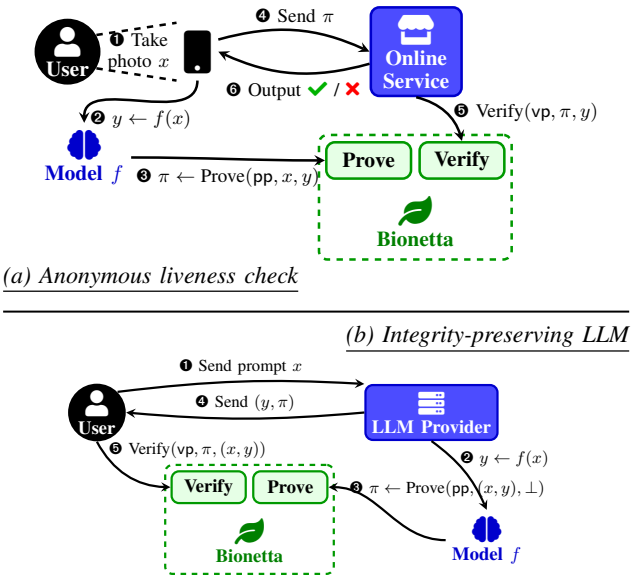

Figure 1: Bionetta applications: anonymous biometric liveness check and ensuring transparent LLM execution.

As of today, the majority of zk-SNARK applications are focused on cryptographic primitives, such as proving knowledge of a hash digest preimage [36], elliptic curve operations [3] or pairings [43]. These applications on their own open doors to a wide range of use cases, including arguably one of the most interesting — national passport verification [74], which has already seen real-world deployment in industry, for example, in the Freedom Tool [72] or ZKPassport [96]. However, this raises the question: can we prove more complex statements?

Consider the following application: an online marketplace wants to integrate an anti-bot protection system through biometrics. Moreover, to secure users' privacy, the biometric data cannot be disclosed during the verification procedure. Suppose developers have trained and published the liveness neural network f [59], [79], taking a piece of biometrics x (e.g., recording of the person moving their head around) and returning whether the person is bona fide or not: $f(x) \in \{0, 1\}$ (with "1" indicating liveness). Then, the user can prove and thus submit the statement:

> *"I am the real person: I have a proof π that I provided the biometrics x and got $f(x) = 1$"*

Case (a) in Figure 1 illustrates this flow. In this scenario, Bionetta ensures two properties: *integrity*, that the verification result is indeed derived by running the claimed neural network f, and *privacy*, that the biometrics x is not revealed to the verifier or any other party.

Another prominent application is ensuring Generative AI integrity [17], [32], [71], [93]. This lets the user verify

that a model provider generated a response correctly *and* with the model corresponding to their subscription tier. In this context, Bionetta can serve as a *Public Verifiable Computation Scheme* that generates the succinct "proof-of-computation" to the user. See case (b) in Figure 1 for a more detailed illustration.

Other notable examples include biometric proximity [70] proof: verifying that the user's biometrics is close to the stored biometrics sample, or proving the creditworthiness of the user without revealing the credit history [86], where the credit score might be calculated by the neural network [88].

Goals. Our study focuses on machine learning applications that process sensitive user data, such as biometric authentication or liveness detection. Thus, to protect user's privacy, the proving procedure must be executed on potentially lightweight client devices. Moreover, to support integration of our solution with decentralized identity applications, such as privacy-preserving attestations [12] or sybil-resilient voting [83], the generated proofs must be effectively verifiable on-chain.

Such specific requirements create technical demands significantly distinct from existing server-side-focused zkML frameworks, such as [71], [93]. Specifically, our design is driven by the following three objectives:

- Goal #1 Proving procedure must be fast and computationally feasible on resource-constrained client devices such as mobile phones.
- Goal #2 Verification procedure and proof sizes must be optimized for efficient on-chain execution (primarily for reducing the gas cost, see § 1.1).
- Goal #3 Proving and verification keys must be compact to minimize client-side storage and on-chain deployment costs, respectively.

1.1. Our Contribution

The main contribution of this paper is **Bionetta** — an R1CS-based *client-side* zkML proving framework that accomplishes all the aforementioned goals. Below we outline the key features of our solution.

Lightweight and fast. Many existing zkML frameworks exhibit an overwhelming proving complexity: for instance, proving MobileNetV2 [75] inference on Intel Xeon E5 takes up to 20 minutes and 45 GB of RAM using the recent zkml framework by Chen et al. [17] or EZKL [82]. Bionetta allows users to construct the same model's circuit using 2.8 million constraints, which can be proven in 23 seconds using 1.8 GB of RAM. We achieve this result by introducing the modification of the Groth16 protocol which we call **UltraGroth**. The primary feature of UltraGroth is *lookup arguments* support which asymptotically reduces the circuit complexity from $O(n)$ to $O(n/\log n)$. We explore this protocol in § 5 and prove its security in Appendix A.

Blockchain Friendly. As motivated by Goal #2, high proving efficiency is not enough. In particular, we want zkML proofs to be verified efficiently on Ethereum Virtual Machine (EVM) compatible chains [89], being the most popular Turing-complete execution engines for smart contracts [38], [91]. Unfortunately, prior zkML frameworks fall short along the three following dimensions:

(a) *large proof sizes*: for example, ezkl produces up to 130 KB-sized proofs while deep-prove [60] up to 4 MB even for simple MNIST [22] models.
(b) *significant verification time*: for instance, while zkCNN [64] or deep-prove [60] have impressive proving efficiency and transparent setup, verification time for even small models with 1 million parameters can reach 1.5 s, while zkml [17] requires up to 12.5 s for the same models.
(c) *large verification keys*: for example, zkml [17] produces verification keys of size up to 2.6 MB while EZKL [82] up to 37 MB.

These three issues make prior zkML frameworks' EVM on-chain verification and deployment largely expensive: for instance, a simple estimate shows that the verification of zkCNN proof for relatively small model LeNet5 [61] costs ≈ 4 million gas[1]. High proof verification cost can potentially be solved with the recursive proofs or proof compression techniques [11], [23] (e.g., sum-check-based protocols such as Ceno [65], used by deep-prove, can be wrapped using Groth16), but this creates significant overhead in the proving time, which becomes especially noticeable for client-side proving. Bionetta paired with UltraGroth does not require any wrappings of the verification procedure and costs only $\approx 300\,\text{k}$ gas.

Generalizability. While the focus of our prior discussion is the usage of UltraGroth for zkML to achieve optimal balance between proving efficiency and on-chain verification cost, we note that Bionetta framework also includes the *R1CS circuit constructor*, explored in § 3 and § 4 (and, as we will see in § 5.4, the multiround R1CS constructor as well). This means that Bionetta can be paired with any zk-SNARKs for R1CS. In particular, if some application utilizing zkML proofs requires transparent setups, one can pair Bionetta's R1CS constructor with, for example, Aurora [9] or Spartan [76].

Client-Side Friendly. All zkML applications mentioned in the previous section can be categorized into two groups: *private weights, public inputs* and *public weights, private inputs*. Most existing zkML frameworks are designed as general-purpose solutions for both categories. To the best of our knowledge, Bionetta is the first zkML framework to tackle them differently. Namely, the second category can be largely optimized by embedding weights of the neural network directly into the relation to be proven. This results in circuit sizes being up to $4\times$ smaller than naively treating each model weight as a separate witness element. While this clearly reduces proving and verification complexities, much less evidently, this drastically decreases verification key size, which for Groth16 and UltraGroth is $O(|\mathbb{x}|)$ where $|\mathbb{x}|$ is the public statement size.

All in all, we summarize all features and system-level trade-offs of Bionetta in Table 1.

1. As reported by zkCNN, the LeNet5's model proof is 63.6 KB, so the calldata cost is already approximately $63600\,\text{bytes} \times 16\,\frac{\text{gas}}{\text{byte}} \approx 1\,\text{mln}$ gas. Since poly-commit proof portion weighs 25.4 KB, the MSM size for BLS12-381 is approximately 530, costing 3.25 million gas. Neglecting the sumcheck verification, we conclude that LeNet5 verification costs more than 4 million gas. This cost increases with larger neural networks.

TABLE 1: Systems-Level Trade-off Analysis. Comparison of Bionetta against GKR-based (e.g., deep-prove [60]) and Halo2-based (e.g., EZKL [82]) frameworks. While Bionetta incurs higher developer friction via per-circuit setups, it is the only solution that satisfies the storage, bandwidth, and cost constraints of lightweight client devices.

Metric	GKR-Based (e.g., zkCNN, Deep-Prove)	Halo2-Based (e.g., EZKL, ZKML)	Bionetta (Ours)
Primary Bottleneck	*Proof Verification*	*Infeasible Proving*	*Compilation Convenience*
Setup Type	✓✓ Transparent	✓ Universal Setup	✗ Per-Circuit Setup
Proving Key Size	✓✓ Not needed	✗✗ Large (∼10 GB – 70 GB)	✓ Tiny (100 MB – 2 GB)
Proving Speed	✓ Fast (seconds)	✗ Very slow (minutes)	✓ Fast (seconds)
Proof Size	✗ Large (100 KB – 4 MB)	✓ Medium (10 KB – 20 KB)	✓✓ Constant (0.32 KB)
On-Chain Verification	✗✗ Prohibitive (≫4M gas)	✗ High (500k – 2M gas)	✓✓ Negligible (200k – 300k gas)
Client-Side Feasibility	✗ Data/Gas Costs	✗ Proving/Storage Limits	✓✓

1.2. Framework Structure

The overall architecture of the *Bionetta* framework is illustrated in Figure 2. The developer flow is as follows:

✓ **Step 1.** First, the developer either trains the neural network using any available framework (e.g., TensorFlow [1] or PyTorch [68]) or loads the existing one. Layers in the model must be supported by Bionetta.

✓ **Step 2.** Next, the developer passes the model architecture and weights to the Bionetta framework, which, under the hood, performs numerous steps:

1) The framework first quantizes the model weights and generates the code in Circom [8] or Noir [5].
2) The framework runs the generated code and obtains the R1CS specification and the witness format specification (e.g., *.sym* file).
3) The framework then builds a custom *witness generator*, optimized specifically based on the arithmetization scheme described in § 4.
4) Bionetta generates the proving key (*.zkey* file). Additionally, if needed, the framework builds bindings for the witness generator.

✓ **Step 3.** Finally, the user can generate proofs by first running the custom witness generator and then passing the generated witness with the proving key to the proving system. The generated proof can be verified on-chain using the autogenerated smart contract.

2. Zero-Knowledge Proofs

We consider various effective relations $\mathcal{R}$, where each pair $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$ consists of a public input (public statement) $\mathbb{x}$ and a private witness $\mathbb{w}$, both of which we refer to as *signals*. The prover wants to convince the verifier of the knowledge of the secret $\mathbb{w}$ for the given public $\mathbb{x}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$. An example relation that is commonly used in blockchain applications [58] is the hash preimage relation $\mathcal{R}_{\text{Hash}} = \{(\mathbb{x}, \mathbb{w}) : \mathbb{x} = H(\mathbb{w})\}$ where H is a hash function.

To prove and further verify such relation, we need to use a zero-knowledge proof protocol. We give the definition of the most widely used type — zk-SNARK.

Definition 2.1. *The **zero-knowledge succinct non-interactive argument of knowledge** (zk-SNARK) is the protocol* $\Pi_{\textit{zk-SNARK}} = (\text{Setup}, \text{Prove}, \text{Verify})$ *where:*

- $\text{Setup}(\mathcal{R}, 1^\lambda) \to (\mathsf{pp}, \mathsf{vp})$ *— the procedure takes the security parameter* $\lambda \in \mathbb{N}$ *and outputs proving and verification keys* pp *and* vp *for the relation* $\mathcal{R}$.
- $\text{Prove}(\mathsf{pp}, \mathbb{x}, \mathbb{w}) \to \pi$ *— the proving algorithm takes the proving key* pp*, public input* $\mathbb{x}$ *and private witness* $\mathbb{w}$ *and outputs the proof* π.
- $\text{Verify}(\mathsf{vp}, \mathbb{x}, \pi) \to \{0, 1\}$ *— the verification algorithm takes the verification key* vp*, statement* $\mathbb{x}$ *and the proof* π*; it either rejects or accepts* π.

*The given protocol must be **complete**, **knowledge sound**, **zero-knowledge** and **succinct**. Succinctness guarantees that the proof size* $|\pi|$ *is sublinear in the size of the witness* $|\mathbb{w}|$ *and the verification time* T *is sublinear in the size of the circuit* $|\mathsf{C}|$*. In particular, hereafter we only consider **strongly succinct** zk-SNARKs, meaning* $|\pi| = \mathcal{O}_\lambda(\log |\mathsf{C}|)$*,* $T = \mathcal{O}_\lambda(|\mathbb{x}|, \log |\mathsf{C}|)$*. For exact technical specifics and definitions, see Appendix A.*

2.1. R1CS and Proving Systems

While for some particular relations there are effective proving systems (for example, for the discrete log relation one might use Schnorr Identification Protocol [52] while for Pedersen commitment one might use Okamoto Protocol [57]), for general NP statements we need to represent the relation in the appropriate arithmetization system. As mentioned in the beginning, we focus on **Rank-1 Constraint System** (R1CS).

We introduce the *extended witness* $\mathbf{z} \in \mathbb{F}^n$, which, informally, contains (a) signals $\mathbb{x}$, $\mathbb{w}$, and (b) all the intermediate computation results of the circuit. R1CS allows us to impose the set of m constraints on the extended witness $\mathbf{z}$, where each constraint is of form $\langle \boldsymbol{\ell}_i, \mathbf{z} \rangle \cdot \langle \boldsymbol{r}_i, \mathbf{z} \rangle = \langle \boldsymbol{o}_i, \mathbf{z} \rangle$ for $i \in [m]$ (we use notation $[m] \triangleq \{0, 1, \ldots, m-1\}$) and constant vectors $\boldsymbol{\ell}_i, \boldsymbol{r}_i, \boldsymbol{o}_i \in \mathbb{F}^n$. More succinctly, the R1CS is written in the matrix form

$$L\mathbf{z} \odot R\mathbf{z} = O\mathbf{z}, \quad L, R, O \in \mathbb{F}^{m \times n}$$

The primary reason why we choose R1CS arithmetization is that in contrast to widely used $\mathcal{P}$lon$\mathcal{K}$ish [31] or

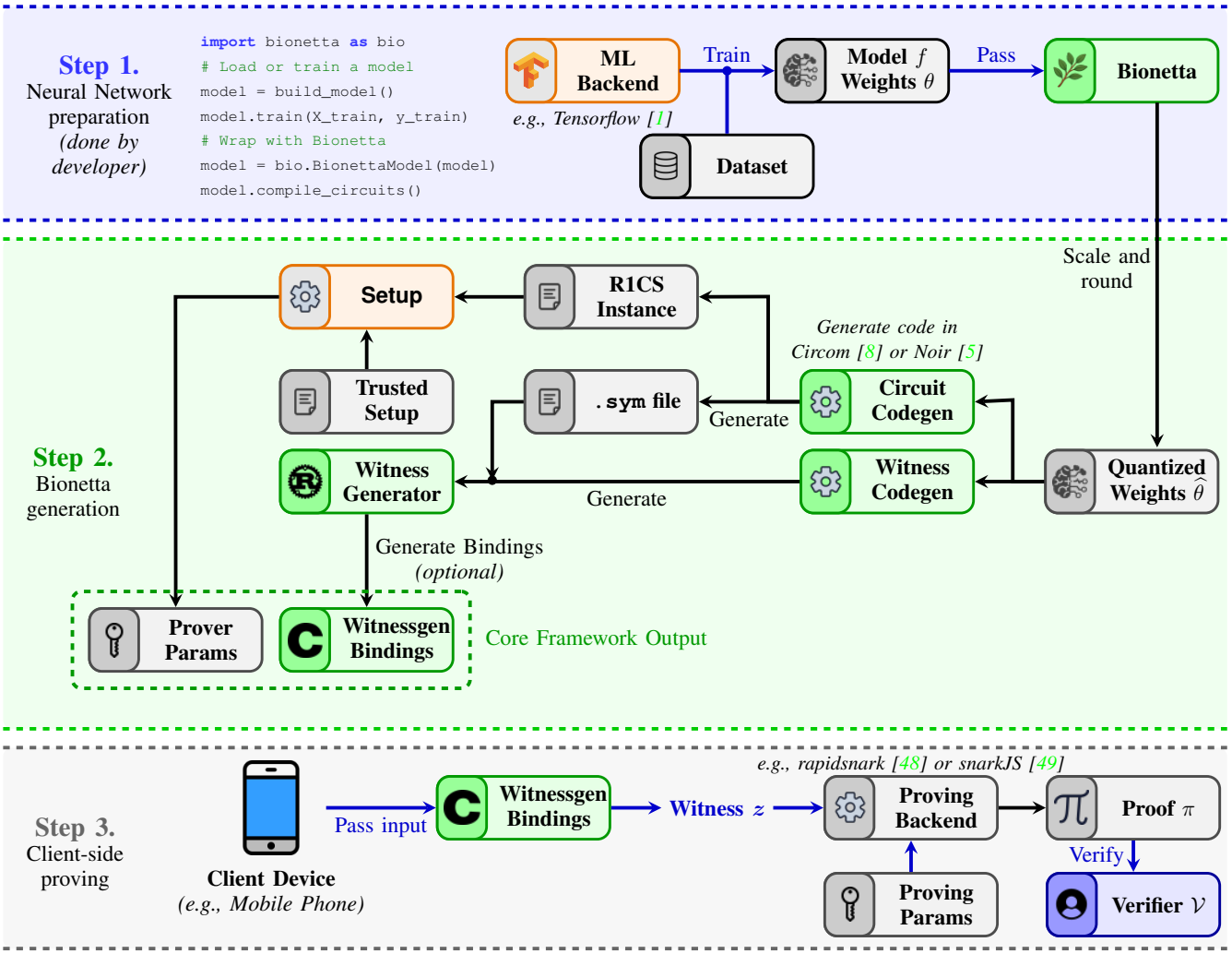

Figure 2: Architecture of the Bionetta framework in three stages: (1) neural network preparation, (2) Bionetta inner workings, and (3) client-side proving. The first step is done by the circuit developer, the second step is done automatically inside the Bionetta framework after running the `.compile_circuits()` method on the prepared model, and the third step is done by the client device using the Bionetta bindings. Note that we do not explicitly specify verifier parameters in the scheme since they can be derived from prover parameters, but they are also generated by Bionetta: for instance, by autogenerating the verifier smart contract.

GKR [35] arithmetization, *computing linear combinations of the signals in R1CS is constraint-free*. This allows to impose the optimization in the subsequent § 3.

Remark. A recent paper on Garuda [21] shows that the cost of linear combinations can be mitigated even for SNARKs over custom gates. However, it requires *circuit-specific setup*, thus removing one of the main benefits of $\mathcal{P}$lon$\mathcal{K}$-ish arithmetization SNARKs. It is an open question whether it is possible to construct transparent SNARK over standard circuits with free linear gates.

While our primary focus is Groth16 and UltraGroth proving systems due to great trade-offs between the prover complexity, verifier complexity, and proof size, our system is fully compatible with any proving system supporting R1CS, such as WHIR [4], Spartan [76], Aurora [9], Fractal [19], Pari [21] or Nova [56] which might give different benefits based on the specific application (such as transparent setup).

Comparison to prior zkML. In Table 2, we specify efficiency metrics of Bionetta (paired with Groth16, UltraGroth, and Spartan), and compare it with several notable state-of-the-art zkML solutions. Notice that previous frameworks fall into two categories: GKR-based approaches and Halo2-based approaches. Our study enables the third category: R1CS-based approaches. For instance, using Bionetta with UltraGroth allows making *proving time sublinear in the total non-linear operations complexity* with constant verification, while pairing it with Spartan allows transparent setup with linear prover and logarithmic verifier. This sharply contrasts with GKR approaches that typically require $O(\sqrt{\eta})$ verifier and proof sizes, violating Goal #2 (where η is the circuit size). While asymptotics of Halo2-based approaches is insignificantly worse than Bionetta's, they require large proving keys of size $O(\eta)$. As we will see in § 6, this can result in proving keys reaching up to 70 GB even for moderately sized neural networks, thus violating Goal #3.

TABLE 2: Complexity comparison of Bionetta with several state-of-the-art zkML frameworks. E denotes the pairing cost, H denotes the hashing cost.

zkML	zkSNARK	Setup		Prover	Verifier	Proof Size
Bionetta	Groth16 [37]	C	$\mathbb{G}_1$	$O(\ell\lambda)$	$3\,E + O(\lvert\mathbb{x}\rvert)$	2
			$\mathbb{G}_2$	$O(\ell\lambda)$		1
			$\mathbb{F}$	$O(\ell\lambda \log \ell\lambda)$	—	—
	UltraGroth *(this paper)*	C	$\mathbb{G}_1$	$O(\ell\lambda/\log \ell\lambda)$	$4\,E + O(\lvert\mathbb{x}\rvert)$	3
			$\mathbb{G}_2$	$O(\ell\lambda/\log \ell\lambda)$		1
			$\mathbb{F}$	$O(\ell\lambda)$	$1\,H$	1
	Spartan [76] & Virgo [94]	T	$\mathbb{F}$	$O(\ell\lambda \log \ell\lambda)$	$O(\log^2 \ell\lambda)\,H$	$O(\log^2 \ell\lambda)$
	Spartan [76] & vSQL [95]	U	$\mathbb{G}$	$O(\ell\lambda)$	$O(\log^2 \ell\lambda)$	$O(\log^2 \ell\lambda)$
			$\mathbb{F}$	$O(\ell\lambda)$	$O(\log^2 \ell\lambda)$	$O(\log^2 \ell\lambda)$
zkCNN [64]	GKR [35] & Hyrax [85]	T	$\mathbb{G}$	$O(n + \ell\rho)$	$O(\sqrt{n + \ell\rho})$	$O(\sqrt{n + \ell\rho})$
			$\mathbb{F}$	$O(\eta + \ell\rho)$	$O(\log(\eta + \ell\rho))$	$O(\log(\eta + \ell\rho))$
zkGPT [71]	GKR [35] & Hyrax [85] & Lasso [77]	T	$\mathbb{G}$	$O(\eta)$	$O(\sqrt{\eta})$	$O(\sqrt{\eta})$
			$\mathbb{F}$	$O(2^\rho + \eta)$	$O(\log \eta)$	$O(\log \eta)$
EZKL [82] ZKML [17]	Halo2 [14] & IPA [15]	T	$\mathbb{G}$	$O(\eta^*)$	$O(\log \eta^*)$	$O(\log \eta^*)$
			$\mathbb{F}$	$O(\eta^* \log \eta^*)$	$O(\log \eta^*)\,H$	$O(\log \eta^*)$
				where $\eta^* = \eta + 2^\rho$		

ℓ: # of non-linear calls (e.g., ReLU) λ: finite field bit-size ($\log_2 |\mathbb{F}|$) ρ: precision parameter
n: # of model parameters $\eta = \mathsf{poly}(n)$: total # of addition and multiplication gates
C: per-circuit setup U: universal setup T: transparent setup

3. Circuit-Embedded Weights

One of our core optimizations is what we call **circuit-embedded weights**. To demonstrate its essence, consider the example of a multiclass linear regression model, which is widely used in Machine Learning — even in traditional facial biometric systems like Eigenface [84] or Fisherface [7]. Such a model is governed by the equation $f(\mathbf{x}; W) := W\mathbf{x}$ with $\mathbf{x} \in \mathbb{R}^n$ and $W \in \mathbb{R}^{m \times n}$. Further assume that we perform all operations over the finite field $\mathbb{F}$: we explain in § 4 how to represent calculations over $\mathbb{R}$ using arithmetic over $\mathbb{F}$. Denote by $\mathtt{C}_f(\mathbf{x}; W)$ the arithmetic circuit representing computation of f over $\mathbb{F}$.

The majority of existing zkML frameworks, such as [17], [26], [60], [66], [82], build the circuit for the following relation that parameterizes the model $\mathtt{C}_f$ (assuming model weights are public and inputs are private):

$$\mathcal{R}^{(f)}_{\text{ZKML}} = \left\{ \begin{matrix} \mathbb{x} = (y_1, \ldots, y_m, W) \\ \mathbb{w} = (x_1, \ldots, x_n) \end{matrix} \;\middle|\; \mathbf{y} = \mathtt{C}_f(\mathbf{x}; W) \right\}.$$

This relation is quite natural: if we re-train the model (thus, this changes the weights W), we pass the new weights to the prover. However, this approach has a significant drawback: since in such case weights are signals, the prover needs to impose constraints on each term $W_{i,j}x_i$ for indices $(i, j) \in [n] \times [m]$, even though W is part of the public protocol. This means that the prover computes the matrix-vector product $W\mathbf{x}$ in $\mathcal{O}(nm)$ constraints. While such complexity is acceptable for small-sized models (especially for simple cases like linear regression), when the model becomes deeper, the number of constraints significantly increases with each such linear operation.

In contrast, we propose to treat the weights W as constants and not as signals. Indeed, consider the following analogy: one writes circuit optimizations for the protocol, requiring computing the elliptic curve scalar product $[\alpha]P$ for *fixed* point $P \in E(\mathbb{F}_q)$. Treating P as a public signal is suboptimal: instead, one can hardcode the point P into the circuit and precompute certain intermediate values: for example, as described in [40], one can precompute powers-of-two $\{[2^j]P\}_{j \in [t]}$ for t-bit sized $\#E(\mathbb{F}_q)$, thus eliminating a significant portion of constraints.

Returning to our problem, formally, instead of building the relation $\mathcal{R}^{(f)}_{\text{ZKML}}$ over the public signals W, we *parameterize* the relation $\mathcal{R}^{(f)}_{\text{Bionetta}}$ over the *constant* weights W:

$$\mathcal{R}^{(f)}_{\text{Bionetta}}(W) = \left\{ \begin{matrix} \mathbb{x} = (y_1, \ldots, y_m) \\ \mathbb{w} = (x_1, \ldots, x_n) \end{matrix} \;\middle|\; \mathbb{x} = \mathtt{C}_f(\mathbb{w}; W) \right\}$$

This way, we first train the neural network, then we extract the weights W, quantize them to $\widehat{W}$, and finally set up the proving system by running $\mathrm{Setup}(1^\lambda, \mathcal{R}^{(f)}_{\text{Bionetta}}(\widehat{W}))$. In engineering terms, this means that we hardcode the weights $\widehat{W}$ into the code. This is illustrated in Figure 3.

Corollary. Any matrix-matrix or matrix-vector multiplications in Bionetta where the matrix is fixed (e.g., Batch Normalization or Dense and Convolutional layers

Algorithm. Circom-like Pseudo-code for the linear regression zero-knowledge circuit

```
circuit LinearRegression(n: int, m: int):
    - public signal input W[n][m];
    + const W[m][n] = [[0x642, 0x123, ...],
    +                  [0x325, 0x786, ...],
    +                  ...];
    private signal input x[n];
    public signal output y[m];

    # Matrix-vector product y = Wx
    for i in 0..m:
        y[i] <== 0;
        for j in 0..n:
            y[i] += W[i][j] * x[j];
```

Figure 3: Optimization by replacing public weight inputs with hardcoded constants.

before applying non-linear activation) *can be implemented in* ≈ 0 *constraints*. Consequently, many classical machine learning algorithms such as *PCA*, *LDA*, *Linear/Logistic Regression*, *Linear SVM* [28] can be implemented in ≈ 0 constraints. However, this comes with significantly increased compilation time since the R1CS compiler must perform the Gauss-Jordan elimination over a significantly larger number of variables. However, we expect that with more advanced optimization techniques such as [2], the compilation time issue can be mitigated.

4. Float Quantization

4.1. Previous Approaches

Further assume that the finite field $\mathbb{F}$ is prime, that is $\mathbb{F} = \mathbb{F}_p$ for a λ-bit prime p (in our case, $\lambda = 254$). In the literature, the quantization of real numbers is done in the following ways.

Study [33] allows to prove the product of two wires x and y with a result z was calculated with the relative error of at most δ: that is, $|z - \mathsf{mul}(x, y)| \leq \delta|\mathsf{mul}(x, y)|$ holds where $\mathsf{mul}(x, y)$ is the true result of the multiplication. It costs roughly 30 constraints per multiplication and 90 constraints per addition.

Study [27] researched application of ZK to location privacy. Since geographical data requires high precision data, the paper proposed the effective lookup-based implementation of the IEEE 754. This allowed to reduce the number of constraints down to 64 per operation.

Most current zkML frameworks like ZEN [29], zkCNN [64], or zkGPT [71] use the quantization scheme where the floating-point number x and its integer representation q are related as $x = S(q - Z)$, where S is the quantization scale while Z is the zero-point integer. This way, given integer representations q_1 and q_2, their integer-represented product is obtained as $q_3 = Z_3 + \frac{S_1 S_2}{S_3}(q_1 - Z_1)(q_2 - Z_2)$. Since $\frac{S_1 S_2}{S_3}$ is a floating-point constant, it is approximated as $2^{-E}\widetilde{e}$, thus requiring performing the shift by E bits. In context of modern ZK systems, this requires full bit-decomposition: for BN254 field [6], for instance, this costs $\approx$254 constraints per shift. Some approaches like zkGPT [71] use lookup checks to avoid such high cost, resulting in 4 constraints per multiplication. However, their approach only works for quantized neural networks where computations are done over 16-bit integers. Extending such approaches to larger integers would result in overwhelming lookup table commitment cost (which is proportional to 2^ρ for ρ bits of precision).

Evidently, the majority of aforementioned approaches require the significant number of constraints to verify the computation correctness. We most closely follow the approach presented in *Mystique* [87], allowing to perform operations with the same cost as the finite field arithmetic, that is: (a) multiplication of two signals costs 1 constraint, (b) addition of two signals/constants costs 0 constraints, (c) multiplication of a signal by a constant costs 0 constraints. Moreover, we mitigate the overflow issue which was neglected in many previous works such as in `keras2circom` [26].

4.2. Quantization Scheme

First, we define how we encode the finite field element into the integer (call it $\mathbf{Z} : \mathbb{F}_p \to \mathbb{Z}$) and how we decode the integer back to the finite field element: call it $\mathbf{F}_p : \mathbb{Z} \to \mathbb{F}_p$. In [87], the mapping $\mathbf{Z}$ is defined as follows: if the field element x is in range $[0, \frac{p-1}{2}]$ (corresponding to approximately the half of the field), we map it to x; otherwise, we assume that x is negative and map it to $x - p$. The inverse mapping $\mathbf{F}_p$ is defined similarly: if the integer x is positive, we map it to x; otherwise, we assume that x is negative and map it to $p + x$.

In our approach we make a tiny adjustment: instead of considering the range $[0, \frac{p-1}{2}]$, we consider the range $[0, 2^{\lambda-1}]$ where λ is the bit-length of p. This is primarily done for efficiency reasons: checking that $x \leq 2^{\lambda-1}$ is a matter of checking a single bit of x, while $x \leq \frac{p-1}{2}$ requires comparing two integers bit-by-bit. Note that such change does not affect the computation correctness much as long as our integers do not exceed $\lambda - 2$ bits. This way, we have the following two maps:

$$\mathbf{F}_p(x) \triangleq \begin{cases} x, & \text{if } 0 \leq x < 2^{\lambda-1} \\ p + x, & \text{if } -2^{\lambda-1} < x < 0 \\ \perp, & \text{otherwise} \end{cases}$$

$$\mathbf{Z}(x) \triangleq \begin{cases} x, & \text{if } 0 \leq x < 2^{\lambda-1} \\ x - p, & \text{if } 2^{\lambda-1} \leq x < p \end{cases}$$

To fully define the quantization scheme, we need a way to map the real number to the integer, which, using the above maps, can be mapped to the finite field and vice versa. Such mapping will be simply defined by multiplication by a large power of two, which we denote by ρ, and the subsequent rounding: $x \mapsto [2^\rho x]$. To dequantize the value y, we will simply divide the integer by the same power of two: $y \mapsto 2^{-\rho} y$. Now, let us combine these two maps to get the quantization and dequantization algorithms.

Definition 4.1. *The* ***Bionetta Quantization Scheme*** *consists of two algorithms: quantize* $Q_\rho : \mathbb{R} \to \mathbb{F}_p$ *and dequantize* $D_\rho : \mathbb{F}_p \to \mathbb{R}$. *They are defined as follows:*

$$Q_\rho(x) \triangleq \mathbf{F}_p([2^\rho x]), \quad D_\rho(\widehat{x}) \triangleq 2^{-\rho}\mathbf{Z}(\widehat{x})$$

*We call $\rho \in \mathbb{N}$ the **base precision parameter**. In practice, the larger ρ is, the more precise the quantization is, yet the trickier it is to handle the overflow: see § 4.3.*

With the above definition, we can now define how we add and multiply two quantized values: say, $\widehat{x} := Q_\rho(x)$ and $\widehat{y} = Q_\rho(y)$ for some reals $x, y \in \mathbb{R}$. The addition is performed by simply adding the quantized values $\widehat{x}$ and $\widehat{y}$. The multiplication is trickier: we first compute the $\widehat{z} \leftarrow \widehat{x} \cdot \widehat{y}$, but how do we dequantize the result? We propose to dequantize $\widehat{z}$ with precision $\rho' := 2\rho$.

Why does this hold? For simplicity, assume $x, y > 0$. Then, the quantized product is $\widehat{z} = Q_\rho(x)Q_\rho(y) = [2^\rho x][2^\rho y]$ which, assuming $x, y \gg 2^{-\rho}$, yields approximately $2^{2\rho}xy$. This way, to get the expression xy back from $\widehat{z}$, we multiply it by $2^{-2\rho}$. It is then straightforward to formally show that such sequence of "*quantization→multiplication→dequantization*" results in the negligible error in ρ: see Appendix D.

This motivates to define operations over quantized values as follows.

Definition 4.2. *Assume $\widehat{x} := Q_{\rho_x}(x)$ and $\widehat{y} := Q_{\rho_y}(y)$ with (generally) different precisions ρ_x and ρ_y. Without loss of generality, assume $\rho_x \le \rho_y$. Then, we define the following operations on the quantized values $\widehat{x}$ and $\widehat{y}$:*

- ***Addition:*** *$\widehat{z} := 2^{\rho_y - \rho_x}\widehat{x} + \widehat{y}$, the output precision is the maximum precision among inputted: $\rho_z := \rho_y$.*
- ***Multiplication:*** *$\widehat{z} := \widehat{x} \cdot \widehat{y}$, the output precision is the sum of input precisions $\rho_z := \rho_x + \rho_y$.*
- ***Multiplication by a constant** c: Precompute the quantized value with the base precision $\widehat{c} := Q_\rho(c)$ and then compute $\widehat{z} := \widehat{x} \cdot \widehat{c}$ with the output precision $\rho_z := \rho_x + \rho$.*

4.3. Overflow Handling

Notice that almost each operation, even multiplication by a constant, increases the precision of the intermediate value at least by ρ. If this precision becomes too-large, we will encounter the overflow, making output of the circuit incorrect. As a simple example, computing x^{300} by writing the circuit $\widehat{x}^{300}$ will fail, since if x was given with precision ρ, the result $\widehat{x}^{300}$ would have 300ρ bits of precision, which easily overflows the field.

Many existing approaches [71] solve this issue by *cutting* the precision of the intermediate values after each operation, that is, by shifting the intermediate value by appropriate number of bits, which, as discussed, is a costly operation. Some approaches like `keras2circom` [26][2] do not solve this issue and assume that the intermediate values will not overflow, which is not the case for most neural networks.

We decided to follow the approach from zkCNN [64] and *cut* the precision of the intermediate values only during specific operations. In fact, we do that only after applying non-linear activations: see Figure 4 for illustration. Indeed, notice that the cost of bit shifting comes primarily from the necessity to impose the constraints on each bit in the bit decomposition of the value. After each layer, we need to apply the non-linearity to the output values: say, compute $\mathbf{y} = \sigma(\mathbf{x}) = (\sigma(x_1), \ldots, \sigma(x_n))$ for some non-linearity $\sigma : \mathbb{R} \to \mathbb{R}$. For certain choices of σ, it turns out that "compute σ, then cut the precision" costs the same as "compute σ" (that is, precision cuts come for free).

2. According to the `keras2circom` documentation, if the neural network consists of ℓ layers, the allowed precision multiplication factor (constant by which all floating-point weights are multiplied) is $10^{76/\ell}$. Since many neural network architectures easily exceed the depth of 20 or 30, this results in a multiplication factor of 340–6300 – too small for the computation error to be negligible.

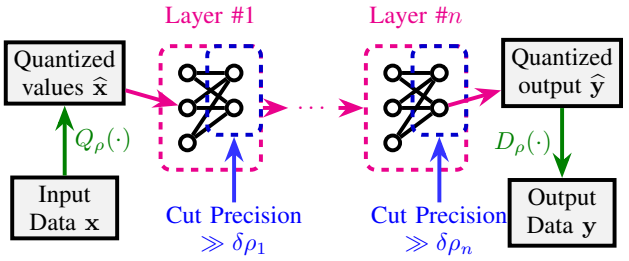

Figure 4: The precision cut operation in the Bionetta system. The dashed magenta boxes represent the layers of the neural network. The dashed blue box represents the output of the layer i, which is cut by $\delta\rho_i$ bits. The green arrows represent the pre- and post-processing of the data done off-circuit.

Implementing ReLU with free precision cut. Let us show that this is the case for $\sigma = \mathrm{ReLU}$. Given $x \in \mathbb{F}_p$ with precision ρ_x, we want to compute $\mathrm{ReLU}(x) = \max\{0, \mathbf{Z}(x)\}$ and then cut the precision down to $\rho < \rho_x$ bits. We first decompose x into bits $x = \sum_{i=0}^{\lambda-1} x_i 2^i$ where we impose λ constraints $x_i(1 - x_i) = 0$ for each $i \in [\lambda]$. Notice that the check "x is negative" is encoded in the element $x_{\lambda-1}$: if $x_{\lambda-1} = 0$, then $x \ge 0$; otherwise, $x < 0$. Thus, without cutting the precision, the output is essentially $(1 - x_{\lambda-1})x$, costing $\lambda + 1$ constraints in total. Our claim is that shifting by $\delta\rho := \rho_x - \rho$ bits (thus, making x of precision ρ) can also be done in $\lambda + 1$ constraints. Indeed, return $(1 - x_{\lambda-1}) \sum_{i=\delta\rho}^{\lambda-1} x_i 2^{i-\delta\rho}$ instead. Note that computing the sum is free in R1CS since all x_i are already constrained.

That said, our strategy for effective quantization is the following: we set the base precision ρ and quantize the input data to the circuit using Q_ρ. Then, we run the circuit and do not cut the precision of the intermediate values until we reach the computation of ReLU-like non-linearities. At that point, given the current precision ρ', we perform the right bit-shift by $\rho' - \rho$ and proceed.

Remark. Note that such approach requires careful selection of precision of each individual weight during the initialization process: consider the linear model $f(\mathbf{x}; W, \boldsymbol{\beta}) := W\mathbf{x} + \boldsymbol{\beta}$ as an example. Fix the base precision ρ. Assuming we quantize both the input $\mathbf{x}$ and matrix W with the same precision ρ, we need to quantize the bias $\boldsymbol{\beta}$ with the precision 2ρ. Indeed, note that the product of $\widehat{\mathbf{x}} = Q_\rho(\mathbf{x})$ and $\widehat{W} = Q_\rho(W)$ has the precision 2ρ. Thus, since the bias is added to the product $\widehat{W} \cdot \widehat{\mathbf{x}}$, it should be quantized with the precision 2ρ as well.

5. UltraGroth: Lookup Tables in R1CS

Let us summarize what we have so far. Notice that due to § 3, all linearities are free. Therefore, the total

circuit size for the neural network consisting of ℓ non-linear operations is approximately $O(\ell\lambda)$. One way to further decrease the number of constraints is to reduce the number of activations ℓ, which we achieve by developing what we call *R1CS-friendly neural network architectures*. We specify the details in Appendix C. However, a more substantial approach is to modify the Groth16 to support lookup tables, which we call *UltraGroth*. Such upgrade allows to reduce the circuit complexity asymptotics to $O(2^w + \frac{n}{w})$ where $w \in \{15, \ldots, 25\}$ in practice. In simple terms, it means that instead of approximately λ constraints per non-linear call, we use only $\frac{\lambda}{w}$ per call with the additional constant overhead of 2^w constraints. Asymptotically, this reduces the prover's work to the sublinear complexity $O(\ell\lambda/\log \ell\lambda)$ compared to Groth16's $O(\ell\lambda)$, which we show in Appendix B.

5.1. Quadratic Arithmetic Programs (QAP)

We first recap the construction of the original Groth16 proving system [37]. Recall that R1CS encodes the program as $L\mathbf{z} \odot R\mathbf{z} = O\mathbf{z}$ for the given fixed matrices $L, R, O \in \mathbb{F}^{m\times n}$ and extended witness $\mathbf{z} \in \mathbb{F}^n$. To make the proof succinct, we need to encode this check in the form of a polynomial identity. Specifically, we construct $3n$ polynomials $\ell_i(X), r_i(X), o_i(X) \in \mathbb{F}^{\leq m}[X], i \in [n]$, interpolating the columns of the corresponding matrices:

$$\ell_i(\omega^j) = L_{i,j},\ r_i(\omega^j) = R_{i,j},\ o_i(\omega^j) = O_{i,j},$$

for every $i \in [n], j \in [m]$ and $\langle\omega\rangle \leq \mathbb{F}^\times$.

The R1CS check can be expressed as the following polynomial identity: the polynomial given by $\mu(X) := \sum_{i\in[n]} z_i\ell_i(X) \cdot \sum_{i\in[n]} z_i r_i(X) - \sum_{i\in[n]} z_i o_i(X)$ has roots in the evaluation domain $\Omega = \{\omega^j\}_{j\in[m]}$. Equivalently, the vanishing polynomial $t(X) = \prod_{\alpha\in\Omega}(X - \alpha)$ must divide the polynomial $\mu(X)$. With the appropriate choice of Ω (namely, primitive roots of 2^t for suitable t), the expression can be simplified down to $t(X) = X^m - 1$ and thus evaluated efficiently.

All in all, we can define the **Quadratic Arithmetic Program** (QAP) relation $\mathcal{R}_{\text{QAP}}$.

Definition 5.1 (QAP)**.** *A **Quadratic Arithmetic Program** is a relation $\mathcal{R}_{QAP}$ parameterized by $3n$ polynomials $\{\ell_i(X)\}_{i\in[n]}, \{r_i(X)\}_{i\in[n]}, \{o_i(X)\}_{i\in[n]}$ over domain Ω. $\mathbf{z} \in \mathbb{F}^n$ is a valid assignment of $\mathcal{R}_{QAP}$ for public statement $\mathbb{x} = \{z_i\}_{i\in I_X}$ and private witness $\mathbb{w} = \{z_i\}_{i\in I_W}$ if:*

$$\sum_{i\in[n]} z_i\ell_i(X) \cdot \sum_{i\in[n]} z_i r_i(X) = \sum_{i\in[n]} z_i o_i(X) + t(X)h(X)$$

for vanishing polynomial $t(X) := \prod_{u\in\Omega}(X - u)$ and some $h(X) \in \mathbb{F}[X]$. Here, n is the total solution witness size, $I_X \subseteq [n]$ is the index set of public inputs, and $I_W := [n] \setminus I_X$ is the index set of private inputs.

5.2. Groth16 Proof Construction

The Groth16 proving system operates over the bilinear group $\mathcal{G} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ with the bilinear pairing function $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. All groups are cyclic and let corresponding generators be g_1, g_2 and g_T, respectively. The field $\mathbb{F}$ is prime: $\mathbb{F} = \mathbb{F}_p$ and the security parameter is $\lambda := |p|$. Now, we specify the implementation of three key procedures of the proving system stated in Definition 2.1: $\mathsf{Setup}(1^\lambda)$, $\mathsf{Prove}(\mathsf{pp}, \mathbb{x}, \mathbb{w})$ and $\mathsf{Verify}(\mathsf{pp}, \mathbb{x}, \pi)$.

Setup(1^λ). Denote by $\zeta_i(X) := \beta\ell_i(X) + \alpha r_i(X) + o_i(X)$. The trusted setup generates the toxic waste $\alpha, \beta, \gamma, \delta, \tau \leftarrow\$ \mathbb{F}^\times$ and computes the common parameters:

$$\mathsf{pp} = \left(g_1^\alpha, g_1^\beta, g_1^\delta, g_2^\beta, g_2^\gamma, g_2^\delta, \left\{g_1^{\tau^i}, g_2^{\tau^i}, g_1^{\tau^i t(\tau)/\delta}\right\}_{i\in[n]}, \left\{g_1^{\zeta_i(\tau)/\gamma}\right\}_{i\in I_X}, \left\{g_1^{\zeta_i(\tau)/\delta}\right\}_{i\in I_W}\right)$$

$$\mathsf{vp} = \left(g_1, g_2, g_2^\gamma, g_2^\delta, g_T^{\alpha\beta}, \left\{g_1^{\zeta_i(\tau)/\gamma}\right\}_{i\in I_X}\right)$$

Prove$(\mathsf{pp}, \mathbb{x}, \mathbb{w})$. The prover samples $r, s \leftarrow\$ \mathbb{F}$ and outputs the proof $\pi = (g_1^{a(\tau)}, g_1^{c(\tau)}, g_2^{b(\tau)})$ where:

$$a(X) = \alpha + \sum_{i\in[n]} z_i\ell_i(X) + r\delta,$$

$$b(X) = \beta + \sum_{i\in[n]} z_i r_i(X) + s\delta,$$

$$c(X) = \delta^{-1}\left(\sum_{i\in I_W} z_i\zeta_i(X) + h(X)t(X)\right) + a(X)s + b(X)r - rs\delta$$

Verify$(\mathsf{pp}, \mathbb{x}, \pi)$. The verifier parses the proof, consisting of three points $\pi = (\pi_A, \pi_C, \pi_B) \in \mathbb{G}_1^2 \times \mathbb{G}_2$ and accepts the proof if and only if the following check holds:

$$e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(\pi_{\text{IC}}, g_2^\gamma) \cdot e(\pi_C, g_2^\delta).$$

Here, we denoted the public input commitment part as $\pi_{\text{IC}} := g_1^{\text{IC}(\tau)} \in \mathbb{G}_1$ with $\text{IC}(X) := \gamma^{-1}\sum_{i\in I_X} z_i \cdot \zeta_i(X)$.

5.2.1. Efficiency Analysis.

Proposition 5.1. *Denote by E_1 and E_2 the cost of exponentiation over $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively. Suppose pairing costs E. Then, the Groth16 proving system complexity is following:*

- *The proving procedure $\mathsf{Prove}(\mathsf{pp}, \mathbb{x}, \mathbb{w})$ runs in the complexity $O(n \cdot E_1 + n \cdot E_2)$.*
- *The verification procedure $\mathsf{Verify}(\mathsf{vp}, \mathbb{x}, \pi)$ requires a time $3E + O(|\mathbb{x}| \cdot E_1)$, while the verification parameters require $O(|\mathbb{x}|)$ elements from $\mathbb{G}_1$.*
- *The proof π consists of 3 group elements: two from $\mathbb{G}_1$ and one from $\mathbb{G}_2$.*

Note that in our case, when the inputs are private, while the weights are public, the vast majority of signals are private, thus $|\mathbb{x}| \ll n$. For that reason, since verification procedure requires only $O(|\mathbb{x}|)$ elements from $\mathbb{G}_1$, the Groth16 verification key size is effective for the on-chain zkML deployment.

5.3. Lookup Tables

The main idea of the UltraGroth system is to utilize lookup tables, which became "native" to the PlonKish [30] and sumcheck-based [39], [81] proving systems. Suppose the values in the lookup tables are $\{t_i\}_{i\in[v]}$ while the part of the witness to be checked is $\{z_i\}_{i\in[n]}$: that is, we prove inclusion $\{z_i\}_{i\in[n]} \subseteq \{t_i\}_{i\in[v]}$. In our particular case, we

will want to have $t_i = i$ for $v = 2^w$ where w is the bit-width of the input. The lookup table will be used to check whether the input is in the range $[0, 2^w)$. One of the most optimal checks was provided in [39]. We briefly restate the key result of the paper.

Theorem 5.1 (Following [39]). *Given two sequences of elements $\{t_i\}_{i\in[v]}$ and $\{z_i\}_{i\in[n]}$, the inclusion check $\{z_i\}_{i\in[n]} \subseteq \{t_i\}_{i\in[v]}$ is satisfied if and only if there exist the set of multiplicities $\{\mu_i\}_{i\in[v]}$ where $\mu_i = \#\{j \in [n] : z_j = t_i\}$ such that:*

$$\sum_{i\in[n]} \frac{1}{X + z_i} = \sum_{i\in[v]} \frac{\mu_i}{X + t_i}$$

In particular, checking such equality at random point from $\mathbb{F}$ results in the soundness error of up to $(n+v)/|\mathbb{F}|$, which becomes negligible for fairly large $|\mathbb{F}|$.

Now, the right hand size of the equation, given that we have sampled the randomness $r \leftarrow\$ \mathbb{F}$, can be computed in v constraints, while the left hand side is computed in additional $2n$ constraints. The caveat here is that Groth16 is not the interactive protocol by default and thus we cannot directly employ the Fiat-Shamir heuristic to sample r. Yet, we do propose a way to sample r by turning the Groth16 into the "interactive-like" proof system. The proceeding *UltraGroth* construction is primarily devoted to correctly and soundly sample randomnesses.

5.4. UltraGroth Construction

To add lookups, we modify Groth16 protocol to add challenge rounds to the proof system, thus generalizing the Mirage protocol [55] which was originally designed for a single round. Specifically, we define the **$(d+1)$-round quadratic arithmetic program (dQAP)** $\mathcal{R}_{\text{dQAP}}$.

Definition 5.2 (Multiround QAP). *$(d+1)$-round QAP is a relation $\mathcal{R}_{dQAP}$ where public statement $\mathbb{x} = \{\mathbb{x}_i\}_{i\in[d+1]}$ and witness $\mathbb{w} = \{\mathbb{w}_i\}_{i\in[d+1]}$ consist of multiple entries, and $\mathbf{z} = (\mathbb{x}_1, \ldots, \mathbb{x}_n, \mathbb{w}_1, \ldots, \mathbb{w}_n) \in \mathcal{R}_{dQAP}$ if*

$$\sum_{i\in[n]} z_i \ell_i(X) \cdot \sum_{i\in[n]} z_i r_i(X) = \sum_{i\in[n]} z_i o_i(X) + t(X)h(X)$$

*and $\mathbb{w}_i = S_i(\mathbb{x}_0, \ldots, \mathbb{x}_i, \mathbb{w}_0, \ldots, \mathbb{w}_{i-1})$ for **strategy** $S = \{S_i\}_{i\in[d+1]}$, consisting of witness generators S_i.*

Here, $\mathbb{w}_i := \{z_j\}_{j\in I_W^{\langle i\rangle}}$ are called *private witnesses* of ith round, $\mathbb{x}_i := \{z_j\}_{j\in I_X^{\langle i\rangle}}$ for $i > 0$ are *challenges* (where we naturally require that $\{I_X^{\langle i\rangle}\}_{i\in[d+1]}$ and $\{I_W^{\langle i\rangle}\}_{i\in[d+1]}$ partition $[n]$), whereas $\mathbb{x}_0$ is a *public input*.

The motivation for introducing *strategies* is as follows: for a multi-round constraint system, we cannot assume that the witness is already known to the prover, as it depends on the verifier challenges. This way, each new witness value is computed from previous witnesses and sampled challenges: $\mathbb{w}_i = S_i(\mathbb{x}_0, \ldots, \mathbb{x}_i, \mathbb{w}_0, \ldots, \mathbb{w}_{i-1})$. As an example, for the lookup constraint system, S_0 computes witness $\mathbb{w}_0$ without imposing lookup checks while strategy S_1 computes $\{\mu_i\}_{i\in[v]}$ and R1CS witness $\mathbb{w}_1$ for the check $\sum_{i\in[n]} \frac{1}{X+z_i} = \sum_{i\in[v]} \frac{\mu_i}{X+t_i}$.

This $(d+1)$-round quadratic arithmetic program represents the following natural interactive protocol between the prover $\mathcal{P}$ and verifier $\mathcal{V}$:

Interactive protocol over $\mathcal{R}_{\text{dQAP}}$

Input: $\mathcal{R}_{\text{dQAP}}$, public input $\mathbb{x}_0$.

For round j from 0 to d:

- If $j > 0$, $\mathcal{V}$ sends a random challenge $\mathbb{x}_j$.
- $\mathcal{P}$ responds with the witness $\mathbb{w}_j$ and if $j = d$, additionally sends $h(X)$.

Final check: Verify $\ell(X)r(X) = o(X) + t(X)h(X)$.

We additionally assume that every public input is bound by an R1CS constraint $a \cdot 1 = a$ with at least one witness value. We impose this condition since UltraGroth protocol performs Fiat-Shamir transformation only over witness signals. Note that the same requirement is also needed for the normal Groth16 protocol in order to prevent malleability.

All in all, we depict the witness structure in Figure 5.

Now we fully describe UltraGroth protocol.

Setup(1^λ). The setup procedure now consists of sampling additional $d+1$ scalars $\delta_0, \ldots, \delta_d \leftarrow\$ \mathbb{F}^\times$ instead of δ. As above, we denote the polynomial linear combination by $\zeta_i(X) := \beta\ell_i(X) + \alpha r_i(X) + o_i(X)$. The modified common parameters are (we highlight changes from Groth16):

$$\begin{aligned}
\mathsf{pp} = \Big(& g_1^\alpha, g_1^\beta, \{g_1^{\delta_i}\}_{i\in[d]}, g_2^\beta, g_2^\gamma, \{g_2^{\delta_i}\}_{i\in[d]}, \\
& \left\{g_1^{\tau^i}, g_2^{\tau^i}, g_1^{\tau^i t(\tau)/\delta_d}\right\}_{i\in[n]}, \\
& \left\{g_1^{\zeta_i(\tau)/\gamma}\right\}_{i\in I_X}, \left\{g_1^{\zeta_j(\tau)/\delta_i}\right\}_{(i,j)\in[d]\times I_W^{(i)}} \Big) \\
\mathsf{vp} = \Big(& g_1, g_2, g_2^\gamma, \{g_2^{\delta_i}\}_{i\in[d]}, g_T^{\alpha\beta}, \left\{g_1^{\zeta_i(\tau)/\gamma}\right\}_{i\in I_X} \Big)
\end{aligned}$$

Prove$(\mathsf{pp}, \mathbb{x}_0, S)$. The prover conducts the steps specified in Algorithm 1.

Verify$(\mathsf{pp}, \mathbb{x}_0, \pi)$. The verifier recomputes the challenges sampled by the prover $\{\mathbb{x}_i\}_{i\in[d+1]\setminus\{0\}}$ and verifies that the corresponding public signals are correct. Then, one checks:

$$e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(\pi_{\text{IC}}, g_2^\gamma) \cdot \prod_{i\in[d+1]} e(\pi_C^{\langle i\rangle}, g_2^{\delta_i})$$

Finally, Theorem 5.2 states the security of the UltraGroth protocol.

Theorem 5.2. *Let $\mathcal{R}_{dQAP}$ be a $(d+1)$-round QAP. In the generic group model (GGM) and random oracle model (ROM), UltraGroth is a zk-SNARK for $\mathcal{R}_{dQAP}$.*

Proof. See Appendix A.

5.4.1. Efficiency Analysis for Bionetta. Let us now analyze the efficiency of the UltraGroth system. First, we specify how UltraGroth helps implement the ReLU function in $O(\lambda/w)$ constraints with a constant $O(2^w)$ commitment cost instead of $O(\lambda)$ described in § 4.3.

Proposition 5.2. *Using UltraGroth, ReLU computation and subsequent right bit shift by a multiple of w can be implemented in $O(\lambda/w)$ constraints (without constant lookup commitment).*

Proof. The circuit for computing $\text{ReLU}(x) \gg \kappa w$ proceeds as follows: let z be the absolute value of x ($z = x$ if $x < 2^{\lambda-1}$ and $z = p - x$ otherwise). To verify

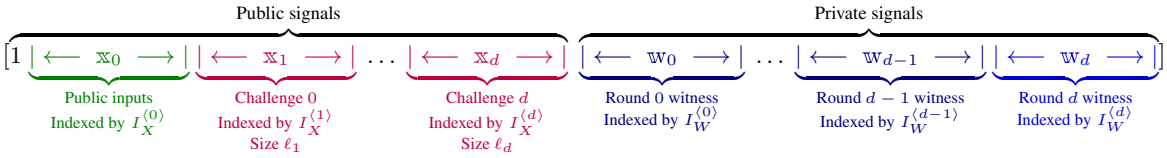

Figure 5: The witness includes three parts: the public part $\mathbb{x}_0$, the private part $\mathbb{w}$ split into $d+1$ rounds $\mathbb{w}_i$ for $i \in [d+1]$ and the sampled challenges $\mathbb{x}_i$ for $i \in [d+1] \setminus \{0\}$.

Algorithm 1: The **Prove**$(\mathsf{pp}, \mathbb{x}_0, S)$ procedure.

```
/* Step 1. Generate witness and sample challenges */
Initialize accumulator a_0 = H(pp)
for round i in [d] do
    r_i ←$ F_p ; /* Sampling a random scalar */
    /* Execute witness gen */
    w_i ← S_i(x_0, ..., x_i, w_0, ..., w_{i-1})
    /* Compute commitment - note that z_j iterates over coordinates of w_i */
    π_C^{⟨i⟩} ← g_1^{c_i(τ)},
      c_i(X) := Σ_{j∈I_W^{⟨i⟩}} z_j ζ_j(X)/δ_i + r_i δ_d
    a_{i+1} ← H(a_i, π_C^{⟨i⟩}) ;   /* Compute accumulator */
    /* Sample x_{i+1} - note that z_j iterates over coordinates of x_{i+1} */
    if i < d: for j ∈ I_X^{⟨i+1⟩} do
        z_j ← H(a_{i+1}, g_1^j)
    end
end
/* Step 2. Generate the proof */
Compute the polynomial h(X) as usual from the QAP check Σ_{i∈[n]} z_i ℓ_i(X) · Σ_{i∈[n]} z_i r_i(X) = Σ_{i∈[n]} z_i o_i(X) + t(X)h(X)
Sample random r, s ←$ F_p and compute π_A ← g_1^{a(τ)}, π_B ← g_2^{b(τ)} and the last commitment π_C^{⟨d⟩} ← g_1^{c_d(τ)} where a(X), b(X), and c_d(X) are defined as follows:
```

$$a(X) = \alpha + \sum_{i \in [n]} z_i \ell_i(X) + r\delta_d,$$

$$b(X) = \beta + \sum_{i \in [n]} z_i r_i(X) + s\delta_d,$$

$$c_d(X) = \frac{1}{\delta_d}\left(\sum_{i \in I_W^{\langle d \rangle}} z_i \zeta_i(X) + h(X)t(X)\right) + a(X)s + b(X)r - \sum_{i \in [d]} r_i \delta_i - rs\delta_d$$

Output: Proof $\pi = (\pi_A, \pi_B, \pi_C^{\langle 0 \rangle}, \ldots, \pi_C^{\langle d \rangle}) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1^{d+1}$

correctness of z, we constrain $(z-x)(z+x) = 0$. Now fix $\ell := \lfloor \frac{\lambda-1}{w} \rfloor$ limbs and compute the decomposition $z = \sum_{j=0}^{\ell} z_j 2^{wj}$ with $0 \le z_j < 2^w$. Note that this ensures that the absolute value $z < 2^{\lambda-1}$. To check that all z_j's are in appropriate range, we use Theorem 5.1, where $X \leftarrow\$ \mathbb{F}$ is sampled by means of UltraGroth. Compute the sign of x as follows: $s = 1$ if $z = x$, and $s = 0$, otherwise. Compute $z' = \sum_{j=\kappa}^{\ell} z_j 2^{(j-\kappa)w}$ to perform the right bit-shift by κw. Finally, output $z' \cdot s$. Such circuit costs $\lfloor (\lambda-1)/w \rfloor + 4 = O(\lambda/w)$ constraints to implement.

Consequently, proving ℓ ReLU calls costs $O(2^w + \frac{\ell\lambda}{w})$ constraints. Such proving efficiency comes with a little cost of one additional pairing and an additional hashing operation. These results were summarized in Table 2.

Note that Bionetta supports not exclusively the ReLU non-linearity. Using the same approach, one can effectively implement: (1) $\mathrm{LeakyReLU}(x) = \max\{\alpha x, x\}$ when $\alpha = 2^{-E}$ is the negative power of two, (2) $\mathrm{ReLU6}(x) = \min\{\mathrm{ReLU}(x), 6\}$, and (3) based on ReLU6, further non-linearities such as *hard-sigmoid*, *hard-swish*, and approximation of GELU. For more details about latter non-linearities, see Appendix C.

Depending on the number ℓ, we choose the chunk parameter w to minimize the cost function $c(w; \ell) = 2^w + \frac{\lambda\ell}{w}$. For optimal $\widehat{w}$, this provides the total complexity of $O(\lambda\ell / \log \lambda\ell)$. We provide details in Appendix B.

6. Experiments

General Benchmarking. To evaluate the performance of aforementioned tools, we benchmark each framework over a set of five neural networks. The first neural network model (which we call the MNIST model) is built with a large number of standard *Dense* (fully connected) and *ReLU* layers to measure baseline zkML frameworks' overhead. This models takes a flattened 28×28 MNIST [22] image as an input and produces a 10-class classification. The architecture consits of two million parameters and 6K activations.

Finally, we employ four other models to test how Bionetta performs with real-world models: we compiled LeNet5 [61], VGG11-mini[3] [80], ResNet18 [42], and MobileNetV2 [75] architectures, which are widely used for various applications such as face recognition [18] or object detection [45].

We carried out tests with the latest available versions of each library [26], [53], [60], [82]. All benchmarks were run on a server with a commodity processor Intel Xeon E5-2665 CPU and 350GB of RAM. For Bionetta, we used TensorFlow 2.12 and our custom fork

3. We reduced VGG11 for benchmarking since `zkml` and EZKL frameworks ran out of RAM memory for original architecture.

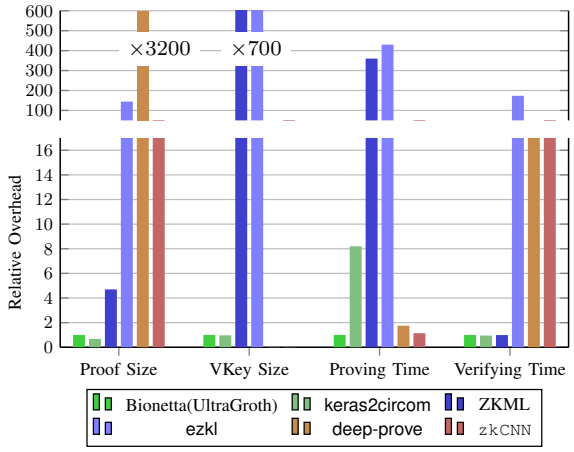

Figure 6: Relative resources overhead of existing zkML frameworks compared to Bionetta (UltraGroth) for a MNIST model with two million parameters. Lower values are better.

of `rapidsnark` [48] prover that supports 1QAP UltraGroth. The results are summarized[4] in Table 3, while the summary for the MNIST model is visualized in Figure 6. As can be seen, Bionetta paired with UltraGroth greatly outperforms all Halo2-based frameworks in terms of all metrics, including proof size (surpassing EZKL by a factor of up to $\times 500$), proving and verification key sizes (up to $\times 9000$-fold reduction), proving (yielding up to a $\times 580$-fold reduction) and verification times (with $\times 1000$ reduction compared to EZKL).

Compared to GKR-based approaches (such as `zkCNN` or `deep-prove`), we expectedly achieve better proof sizes ($\times 40$ improvement compared to `zkCNN` and $\times 4200$ compared to `deep-prove`) and verification times ($\times 72.5$ boost). For proving time, the performance of Bionetta remains on par with such frameworks, indicating no significant drop while still offering substantial gains elsewhere. In Figure 7, we consider more systematically how proving time scales with the model increase.

Moreover, compared to existing solutions, these results indicate that using Bionetta is practical for many potential blockchain applications: recall that UltraGroth proof verification requires only 4 pairing operations while verification keys are of size up to 5KB. That said, our approach successfully achieves all three objectives outlined in § 1: Goal #1, Goal #2, and Goal #3.

Client-side vs Server-side proving. We examine how optimizations introduced in § 3 reduce circuit sizes. We compiled all aforementioned models and recorded the number of constraints and proving efficiency. Results are depicted in Table 4. Expectedly, the decrease in the number of constraints largely depends on the neural network architecture: for instance, UltraGroth circuit size decreases $\times 3$ for LeNet5 circuit while for ResNet18 this optimization yields $\times 32$ improvement. For the latter case, this reduces proving time from 270 seconds down to only 15.

Efficiency Metrics Behaviour. We additionally provide benchmarks for dependency of key efficiency metrics from the number of non-linearities in the neural network. For this purpose, we define the following fully connected neural network:

$$f(\mathbf{x}; W_1, W_2) = W_2 \mathrm{ReLU}(W_1 \mathbf{x}),$$
$$\text{with } \mathbf{x} \in \mathbb{R}^{784}, W_1 \in \mathbb{R}^{N \times 784}, W_2 \in \mathbb{R}^{10 \times N},$$

where we vary the parameter N — number of neurons in the hidden layer. The results for 12 different values of N in range $500 \leq N \leq 6000$ are depicted in Figure 7. Bionetta's verification time and proof size are consistently the best across all values of N. For proving time and RAM consumption with $N < 2000$, the Bionetta gives slightly worse performance than GKR-based approaches. However, as the model becomes larger (starting from roughly 2 million parameters), the Bionetta surpasses these frameworks. For instance, when the model reaches 4.7 million parameters, zkCNN generates proof in 10 seconds using 3.6 GB of RAM while Bionetta in 5.3 seconds consuming only 2.2 GB of RAM.

Efficiency on Mobile Devices. To further demonstrate that Bionetta is capable of proving neural networks on lightweight client devices, we conduct an experiment where we launch all five compiled models on *iPhone 14 Pro* (A16 Bionic chip, 6-core CPU with 2 performance and 4 efficiency cores, 5-core GPU, and 6 GB of RAM). We depict the results in Table 5. For Groth16, we use both the `rapidsnark` [48] and recently published mobile-first proving framework `impl` [50] by Ingonyama. As can be seen, the MobileNetV2 neural network generates the proof in 23 seconds while ResNet requires only 14 seconds. While the peak RAM consumption is considerable (approximately 1.7GB), it is still a doable number for modern edge devices.

Accuracy. Finally, we estimate the accuracy of Bionetta. We launch every model on 10^5 randomly generated inputs (for example, by sampling each input from standard normal distribution) both in TensorFlow and through arithmetical circuit. Then, we dequantize the circuit result and compare it to one obtained in TensorFlow. Assuming that we use precision ρ, the output of the TensorFlow is $\mathbf{y}$ and dequantized result is $\mathbf{y}_\rho$, we measure the relative error as $\varepsilon_\rho = \|\mathbf{y} - \mathbf{y}_\rho\| / \|\mathbf{y}\|$. After running experiments on each model, the maximal observed relative errors among all models are $\varepsilon_{15} \approx 5.2 \cdot 10^{-3}$, $\varepsilon_{30} \approx 3.7 \cdot 10^{-5}$, $\varepsilon_{45} \approx 2.0 \cdot 10^{-6}$, $\varepsilon_{60} \approx 9.4 \cdot 10^{-7}$, which can be considered negligible in practical applications.

7. Limitations

Security considerations. While the current version of *Bionetta* is already capable of proving the inference correctness and does it correctly, soundly, and in zero-knowledge, the state of the current machine learning security research cannot guarantee the *cryptographic* (say, 100-bit) soundness of the neural network predicion itself. Note that there is still a lot of research to be done in the field to ensure that the user cannot attack neural network systems by "grinding" the input data x to find the one that would yield desired output y (such attacks are called White-/Black-boxed attacks and are demonstrated in [16], [62], [69]. In case of biometrics in [13], [54]).

4. Unfortunately, some frameworks like `keras2circom` [26] or `deep-prove` [60] do not support complex architectures like ResNet18 or MobileNetV2, so we do not include them in this table. Yet, we further include `deep-prove` in Figure 7.

TABLE 3: Benchmark results for four frameworks on an Intel Xeon E5-2665 CPU (16 threads @ 3.10GHz) and 350GB of RAM. We present five key metrics: proof size, proving key (PK) size, verification key (VK) size, proving time, and verification time. Colors indicate performance ranking: best, worst.
Notes:
"✗" indicates that the zkML framework does not support all operations used in the given model.
†`zkCNN` is GKR-based and therefore does not require proving and verification keys.

Framework	Metric	MNIST	LeNet5	VGG11-mini	ResNet18	MobileNetV2
Bionetta+UltraGroth	Proof Size (KB)	0.88	0.88	0.88	0.88	0.88
	PK (GB)	0.20	0.22	0.63	1.00	2.41
	VK (MB)	0.004	0.004	0.004	0.004	0.004
	Prove (s)	3.05	3.75	7.70	14.10	24.50
	RAM (GB)	0.27	0.28	0.48	0.75	1.80
	Verify (s)	0.010	0.010	0.020	0.015	0.018
`ezkl` [82]	Proof Size (KB)	127.0	127.0	478.0	180.0	175.0
	PK (GB)	8.30	2.10	17.00	34.00	76.00
	VK (MB)	4.10	1.10	8.10	17.00	37.00
	Prove (s)	1310	535	2650	6840	14320
	RAM (GB)	21.15	5.45	41.85	103.50	203.95
	Verify (s)	5.40	2.00	9.70	25.30	40.60
`zkml` [17]	Proof Size (KB)	5.05	6.70	6.70	6.70	5.10
	PK (GB)	16.10	2.40	19.30	19.30	17.70
	VK (MB)	2.60	0.40	3.15	3.15	2.60
	Prove (s)	1100	185	1580	1660	1135
	RAM (GB)	39.95	6.85	51.95	52.30	44.05
	Verify (s)	0.012	0.013	0.022	0.023	0.020
`zkCNN`† [64]	Proof Size (KB)	23.25	38.10	42.60	✗	✗
	Prove (s)	3.45	1.05	4.10	✗	✗
	RAM (GB)	1.00	0.15	0.80	✗	✗
	Verify (s)	1.00	0.30	1.45	✗	✗

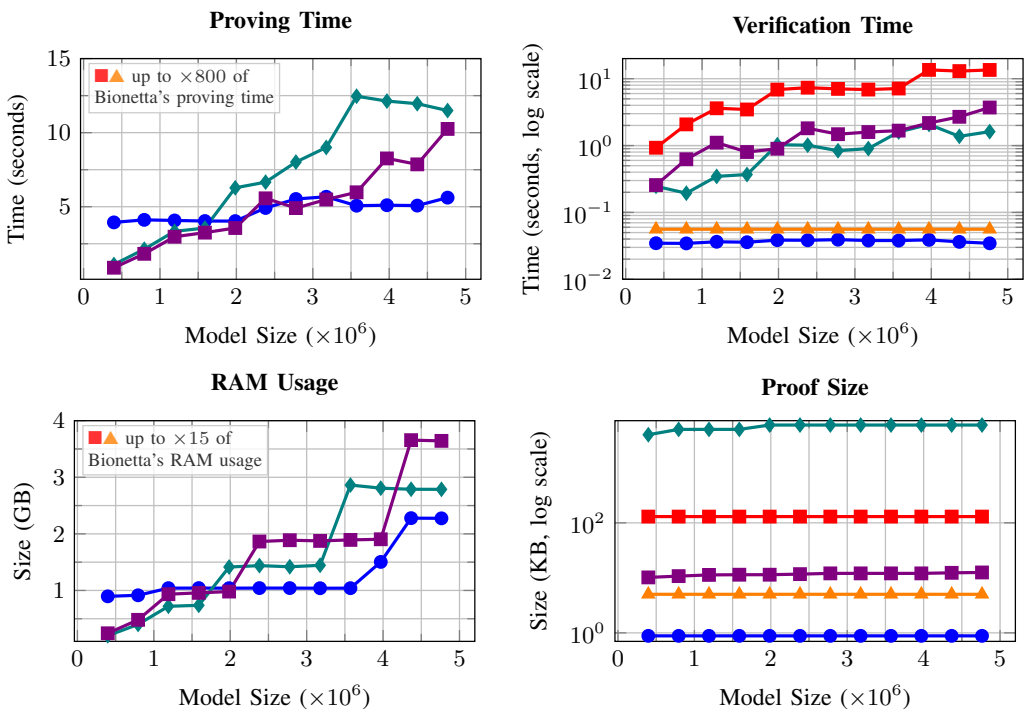

Figure 7: Comparison of proving time, verification time, RAM usage, and proof size across different zkML frameworks as a function of the model's size. *Legend:* ● UltraGroth, ■ EZKL, ▲ ZKML, ◆ `deep-prove`, and ■ zkCNN.

TABLE 4: Comparison of circuits complexities for client-side (marked as "Client" below) and server-side (marked as "Server") provings over QAP and 1QAP. Values for Server-Side proving time and RAM are estimated based on constraints number. In orange we color the most optimal parameters used in Bionetta.

†Values for server-side proving are estimated based on the constraints number since Bionetta is primarily designed for client-side proving.

Mode	Metric	MNIST	LeNet5	VGG11	ResNet18	MobNetV2
1QAP Client	Constraints ($\times 10^6$)	0.25	0.26	0.73	1.16	2.78
	Prove (s)	3.05	3.75	7.70	14.10	24.50
	RAM (GB)	0.27	0.28	0.48	0.75	1.80
1QAP Server†	Constraints ($\times 10^6$)	2.19	0.65	4.17	37.85	7.97
	Prove (s)	15	6	30	270	60
	RAM (GB)	1.2	0.4	2.2	19.8	4.2
QAP Client	Constraints ($\times 10^6$)	1.57	1.69	7.67	12.0	31.8
	Prove (s)	8.30	8.70	56.4	87.5	230.0
	RAM (GB)	0.58	0.60	4.05	6.3	16.65
QAP Server†	Constraints ($\times 10^6$)	3.51	2.08	11.1	49.0	37.8
	Prove (s)	25	15	80	350	270
	RAM (GB)	1.9	1.1	5.8	25.5	19.7

TABLE 5: Bionetta Benchmarks (proving time and peak RAM consumption) on iPhone 14 Pro for both UltraGroth and Groth16. "✗" mark indicates that too much RAM was required to launch the proving procedure. For Groth16, we specify both `rapidsnark` [48] and `impl` [50] benchmarks.

	UltraGroth		Groth16 (`rapidsnark`/`impl`)	
Model	**Time (s)**	**RAM (MB)**	**Time (s)**	**RAM (MB)**
MNIST	2.80	230	8.2/5.0	570/540
LeNet5	3.55	145	7.7/4.80	600/560
VGG11-mini	7.65	465	52.5/36.5	3800/3550
ResNet18	13.60	700	✗/✗	✗/✗
MobileNetV2	22.96	1700	✗/✗	✗/✗

One of the solutions is to use trusted device: see WorldID project [90], for instance. Here, together with an input x, one can send the signature or proof that x was recorded using trusted device. Moreover, in case of face recognition, one may even use the passport's photo and attach the zero-knowledge proof that the photo corresponds to the valid passport: see Rarimo [73] or ZKPassport [96].

Finally, it is worth mentioning the recent paper [34] which shows the secure way of implementing Advanced Encryption Standard (AES) block cipher as a dense neural network. If this concept could be extended to implementing the cryptographically secure neural network inference, then we could use any client-side machine learning proofs in a similar way how we use hash functions or digital signatures in the classical cryptography.

Compilation time. As discussed in § 3, we heavily rely on linear layers optimization: all constant linear combinations of signals are free. However, during conversion of the (Circom) program to the R1CS instance, one needs to conduct a costly Gauss-Jordan elimination algorithm over a large set of variables. An unfortunate consequence is that compilation time of Bionetta is significant. For instance, compilation of ResNet18 using UltraGroth took *1 hour*. Surprisingly to us, seemingly larger neural network MobileNetV2 takes only *20 minutes* to compile. To reduce compilation time, we do not conduct the full simplification (`--o2` flag in Circom), but use the basic simplification (`--o1` flag) instead. However, as a result, not all linear constraints get eliminated: for instance, as reported in Table 4, the MobileNetV2 circuit comprises 2.8 million constraints, among which 0.3 million are linear. That said, our results in Table 5 can be slightly improved by more careful and optimized implementation of linear constraints elimination.

Fortunately, compilation procedure is done only once during the protocol setup, so we believe this is a fair trade-off to get a high client-side proving efficiency.

8. Conclusion

In this study, we presented *Bionetta* — the effective client-side zkML framework. In our framework, we eliminated all linear constraints, implemented effective arithmetization system, and upgraded Groth16 protocol to *UltraGroth* that enabled lookup tables. As a result, our framework surpasses most existing zkML approaches in "client-side mode" by providing tiny constant-sized proofs, constant verification time, small verification keys, and effective prover due to efficient circuit implementation. Moreover, our framework allowed to prove relatively large neural networks (such as *MobileNetV2*) even on mobile phones. We have shown aforementioned achievements by conducting an extensive experimental setup, where we compiled five different large neural networks and compared our approach with prominent solutions such as ZKML, EZKL, zkCNN, and deep-prove. We supplement our findings with the source code that includes implementation of model-to-circuit converter, effective witness generator, *UltraGroth* protocol for $d = 1$, and Swift SDK that enables running proving procedure on iOS platform (see *Open Science* section for the link).

9. Open Science

We leave anonymous artifact (including source code, circuits, UltraGroth implementation for $d = 1$) below:

https://anonymous.4open.science/r/Bionetta-9853

Ethical Considerations

Bionetta is a framework for verifiable client-side computation that allows sensitive data to be processed locally, reducing privacy risks associated with sending information to centralized providers. It ensures computation integrity by enabling users to prove that model's outputs are correct and unmodified. The framework also strengthens transparency by ensuring that the intended model with known, auditable properties is the one actually executed, aligning with the vision of open-source AI ecosystems (such as DeepSeek [10] or Grok [92]). By enforcing determinism, Bionetta allows decentralized applications to rely solely on predefined verification logic without trusted APIs, oracles, or identity providers. At the same time, its flexibility introduces responsibilities: engineers must ensure they respect the licenses of the models they wrap, since Bionetta cannot enforce or verify usage permissions.

References

[1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.

[2] Elvira Albert, Marta Bellés-Muñoz, Miguel Isabel, Clara Rodríguez-Núñez, and Albert Rubio. Distilling constraints in zero-knowledge protocols. In Sharon Shoham and Yakir Vizel, editors, *Computer Aided Verification*, pages 430–443, Cham, 2022. Springer International Publishing.

[3] Diego F. Aranha, Youssef El Housni, and Aurore Guillevic. A survey of elliptic curves for proof systems. *Des. Codes Cryptography*, 91(11):3333–3378, December 2022.

[4] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–solomon proximity testing with super-fast verification. Cryptology ePrint Archive, Paper 2024/1586, 2024.

[5] Aztec. Noir. https://github.com/noir-lang/noir, 2025.

[6] Paulo S. L. M. Barreto and Michael Naehrig. Pairing-friendly elliptic curves of prime order. In Bart Preneel and Stafford Tavares, editors, *Selected Areas in Cryptography*, pages 319–331, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.

[7] P.N. Belhumeur, J.P. Hespanha, and D.J. Kriegman. Eigenfaces vs. fisherfaces: recognition using class specific linear projection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(7):711–720, 1997.

[8] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. Circom: A circuit description language for building zero-knowledge applications. *IEEE Transactions on Dependable and Secure Computing*, 20(6):4733–4751, 2023.

[9] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for r1cs. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019*, pages 103–128, Cham, 2019. Springer International Publishing.

[10] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024.

[11] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for SNARKs and proof-carrying data. Cryptology ePrint Archive, Paper 2012/095, 2012.

[12] Marina Blanton and Dennis Murphy. Privacy preserving biometric authentication for fingerprints and beyond. In *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy*, CODASPY '24, page 367–378, New York, NY, USA, 2024. Association for Computing Machinery.

[13] Philip Bontrager, Aditi Roy, Julian Togelius, Nasir Memon, and Arun Ross. Deepmasterprints: Generating masterprints for dictionary attacks via latent variable evolution. In *2018 IEEE 9th International Conference on Biometrics Theory, Applications and Systems (BTAS)*, page 1–9. IEEE Press, 2018.

[14] Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Paper 2019/1021, 2019.

[15] Benedikt Bunz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. Proofs for inner pairing products and applications. In *Advances in Cryptology – ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part III*, page 65–97, Berlin, Heidelberg, 2021. Springer-Verlag.

[16] Nicholas Carlini and David Wagner. Adversarial examples are not easily detected: Bypassing ten detection methods. In *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, AISec '17, page 3–14, New York, NY, USA, 2017. Association for Computing Machinery.

[17] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. Zkml: An optimizing system for ml inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*, EuroSys '24, page 560–574, New York, NY, USA, 2024. Association for Computing Machinery.

[18] Sheng Chen, Yang Liu, Xiang Gao, and Zhen Han. Mobilefacenets: Efficient cnns for accurate real-time face verification on mobile devices. In *Chinese conference on biometric recognition*, pages 428–438. Springer, 2018.

[19] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. Cryptology ePrint Archive, Paper 2019/1076, 2019.

[20] Grigorios G Chrysos, Stylianos Moschoglou, Giorgos Bouritsas, Yannis Panagakis, Jiankang Deng, and Stefanos Zafeiriou. P-nets: Deep polynomial neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7325–7335, 2020.

[21] Michel Dellepere, Pratyush Mishra, and Alireza Shirzad. Garuda and pari: Faster and smaller SNARKs via equifficient polynomial commitments. Cryptology ePrint Archive, Paper 2024/1245, 2024.

[22] Li Deng. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6):141–142, 2012.

[23] Sai Deng and Bo Du. zkTree: A zero-knowledge recursion tree with ZKP membership proofs. Cryptology ePrint Archive, Paper 2023/208, 2023.

[24] Tim Dokchitser and Alexandr Bulkin. Zero knowledge virtual machine step by step. Cryptology ePrint Archive, Paper 2023/1032, 2023.

[25] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.

[26] drCathieSo.eth. keras2circom. https://github.com/socathie/keras2circom, 2022.

[27] Jens Ernstberger, Chengru Zhang, Luca Ciprian, Philipp Jovanovic, and Sebastian Steinhorst. Zero-Knowledge Location Privacy via Accurate Floating-Point SNARKs . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3440–3459, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.

[28] Johann Faouzi and Olivier Colliot. Classic machine learning methods. *Machine Learning for Brain Disorders*, pages 25–75, 2023.

[29] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. ZEN: An optimizing compiler for verifiable, zero-knowledge neural network inferences. Cryptology ePrint Archive, Paper 2021/087, 2021.

[30] Ariel Gabizon and Zachary J. Williamson. plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive, Paper 2020/315, 2020.

[31] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical non-interactive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.

[32] Bianca-Mihaela Ganescu and Jonathan Passerat-Palmbach. Trust the process: Zero-knowledge machine learning to enhance trust in generative ai interactions, 2024.

[33] Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Yinuo Zhang. Succinct zero knowledge for floating point computations. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, page 1203–1216, New York, NY, USA, 2022. Association for Computing Machinery.

[34] David Gerault, Anna Hambitzer, Eyal Ronen, and Adi Shamir. How to securely implement cryptography in deep neural networks. Cryptology ePrint Archive, Paper 2025/288, 2025.

[35] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: Interactive proofs for muggles. *J. ACM*, 62(4), September 2015.

[36] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *USENIX Security Symposium*, 2021.

[37] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016*, pages 305–326, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.

[38] Huaqun Guo and Xingjie Yu. A survey on blockchain technology and its security. *Blockchain: Research and Applications*, 3(2):100067, 2022.

[39] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.

[40] Darrel Hankerson, Alfred J Menezes, and Scott Vanstone. *Guide to elliptic curve cryptography*. Springer Science & Business Media, 2004.

[41] Meng Hao, Hanxiao Chen, Hongwei Li, Chenkai Weng, Yuan Zhang, Haomiao Yang, and Tianwei Zhang. Scalable zero-knowledge proofs for non-linear functions in machine learning. In *Proceedings of the 33rd USENIX Conference on Security Symposium*, SEC '24, USA, 2024. USENIX Association.

[42] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[43] Youssef El Housni. Pairings in rank-1 constraint systems. In Mehdi Tibouchi and XiaoFeng Wang, editors, *Applied Cryptography and Network Security*, pages 339–362, Cham, 2023. Springer Nature Switzerland.

[44] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019.

[45] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[46] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141, 2018.

[47] Chengpeng Huang, Rui Song, Shang Gao, Yu Guo, and Bin Xiao. Data availability and decentralization: New techniques for zk-rollups in layer 2 blockchain networks, 2024.

[48] iden3. rapidsnark. https://github.com/iden3/rapidsnark, 2025.

[49] iden3. snarkjs. https://github.com/iden3/snarkjs, 2025.

[50] Ingonyama. imp1. https://github.com/ingonyama-zk/imp1, 2025.

[51] Takumi Ishiyama, Takuya Suzuki, and Hayato Yamana. Highly accurate cnn inference using approximate activation functions over homomorphic encryption. In *2020 IEEE International Conference on Big Data (Big Data)*, pages 3989–3995. IEEE, 2020.

[52] Mike Just. *Schnorr Identification Protocol*, pages 1083–1083. Springer US, Boston, MA, 2011.

[53] Daniel Kang. ddkang/zkml framework. https://github.com/ddkang/zkml, 2022.

[54] Yasmeen M. Khedr, Xin Liu, Haobo Lu, and Kun He. Transferable adversarial attacks against face recognition using surrogate model fine-tuning. *Applied Soft Computing*, 174:112983, 2025.

[55] Ahmed Kosba, Dimitrios Papadopoulos, Charalampos Papamanthou, and Dawn Song. Mirage: succinct arguments for randomized algorithms with applications to universal zk-snarks. In *Proceedings of the 29th USENIX Conference on Security Symposium*, SEC'20, USA, 2020. USENIX Association.

[56] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. Cryptology ePrint Archive, Paper 2021/370, 2021.

[57] Lukasz Krzywiecki and Miroslaw Kutylowski. Security of okamoto identification scheme: a defense against ephemeral key leakage and setup. In *Proceedings of the Fifth ACM International Workshop on Security in Cloud Computing*, SCC '17, page 43–50, New York, NY, USA, 2017. Association for Computing Machinery.

[58] Oleksandr Kurbatov, Dmytro Zakharov, Anton Levochko, Kyrylo Riabov, and Bohdan Skriabin. Multichain taprootized atomic swaps: Introducing untraceability through zero-knowledge proofs, 2024.

[59] Oleksandr Kuznetsov, Dmytro Zakharov, Emanuele Frontoni, and Andrea Maranesi. Attacknet: Enhancing biometric security via tailored convolutional neural network architectures for liveness detection. *Computers & Security*, 141:103828, June 2024.

[60] Lagrange Labs. deep-prove. https://github.com/Lagrange-Labs/deep-prove, 2025.

[61] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[62] Huichen Li, Xiaojun Xu, Xiaolu Zhang, Shuang Yang, and Bo Li. Qeba: Query-efficient boundary-based blackbox attack. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1221–1230, 2020.

[63] Junkai Liang, Daqi Hu, Pengfei Wu, Yunbo Yang, Qingni Shen, and Zhonghai Wu. SoK: Understanding zk-SNARKs: The gap between research and practice. Cryptology ePrint Archive, Paper 2025/172, 2025.

[64] Tianyi Liu, Xiang Xie, and Yupeng Zhang. zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 2968–2985, New York, NY, USA, 2021. Association for Computing Machinery.

[65] Tianyi Liu, Zhenfei Zhang, Yuncong Zhang, Wenqing Hu, and Ye Zhang. Ceno: Non-uniform, segment and parallel zero-knowledge virtual machine. Cryptology ePrint Archive, Paper 2024/387, 2024.

[66] Hidde Lycklama, Alexander Viand, Nikolay Avramov, Nicolas Küchler, and Anwar Hithnawi. Artemis: Efficient commit-and-prove snarks for zkml, 2024.

[67] Eduardo Morais, Tommy Koens, Cees van Wijk, and Aleksei Koren. A survey on zero knowledge range proofs and applications. *SN Applied Sciences*, 1(8):946, 2019.

[68] A Paszke. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*, 2019.

[69] Li Pengcheng, Jinfeng Yi, and Lijun Zhang. Query-efficient black-box attack by active learning. In *2018 IEEE International Conference on Data Mining (ICDM)*, pages 1200–1205, 2018.

[70] Sameeksha Prasad, Namita Tiwari, and Meenu Chawla. Zero-knowledge proofs in biometric authentication systems: A review. In Jagdish Chand Bansal, Harish Sharma, and Antorweep Chakravorty, editors, *Congress on Smart Computing Technologies*, pages 279–290, Singapore, 2024. Springer Nature Singapore.

[71] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. zkgpt: An efficient non-interactive zero-knowledge proof framework for llm inference. *USENIX Security 2025, Prepublication*.

[72] Rarimo. Freedom tool whitepaper. https://freedomtool.org/, 2025.

[73] Rarimo. Rarimo: Permissionless (zk) registries. https://github.com/rarimo, 2025.

[74] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. zk-creds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure . In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 790–808, Los Alamitos, CA, USA, May 2023. IEEE Computer Society.

[75] Mark Sandler, Andrew G. Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4510–4520, 2018.

[76] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. Cryptology ePrint Archive, Paper 2019/550, 2019.

[77] Srinath Setty, Justin Thaler, and Riad Wahby. Unlocking the lookup singularity with lasso. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 180–209, Cham, 2024. Springer Nature Switzerland.

[78] Cosimo Sguanci, Roberto Spatafora, and Andrea Mario Vergani. Layer 2 blockchain scaling: a survey, 2021.

[79] Swapnil R. Shinde, Anupkumar M. Bongale, Deepak Dharrao, and Sudeep D. Thepade. An enhanced light weight face liveness detection method using deep convolutional neural network. *MethodsX*, 14:103229, 2025.

[80] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.

[81] Lev Soukhanov. Logup*: faster, cheaper logup argument for small-table indexed lookups. Cryptology ePrint Archive, Paper 2025/946, 2025.

[82] Tobin South, Alexander Camuto, Shrey Jain, Shayla Nguyen, Robert Mahari, Christian Paquin, Jason Morton, and Alex 'Sandy' Pentland. Verifiable evaluations of machine learning models using zksnarks, 2024.

[83] Nguyen Tran, Bonan Min, Jinyang Li, and Lakshminarayanan Subramanian. Sybil-resilient online content voting. pages 15–28, 01 2009.

[84] M.A. Turk and A.P. Pentland. Face recognition using eigenfaces. In *Proceedings. 1991 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 586–591, 1991.

[85] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. Cryptology ePrint Archive, Paper 2017/1132, 2017.

[86] Chien-Chiang Wang and Yiting Li. Anonymous credit. MPRA Paper 118480, University Library of Munich, Germany, March 2023.

[87] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 501–518, 2021.

[88] David West. Neural network credit scoring models. *Computers & Operations Research*, 27(11):1131–1152, 2000.

[89] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger.

[90] World. A new identity and financial network. https://whitepaper.world.org/, 2025. Accessed: 2025-08-08.

[91] Kaidong Wu, Yun Ma, Gang Huang, and Xuanzhe Liu. A first look at blockchain-based decentralized applications. *Software: Practice and Experience*, 51(10):2033–2050, 2021.

[92] xAI. Grok-2 Open-Weights Model. https://huggingface.co/xai-org/grok-2, 2024. Released under the Grok 2 Community License Agreement for non-commercial and research purposes, accessed August 25, 2025.

[93] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. zkPyTorch: A hierarchical optimized compiler for zero-knowledge machine learning. Cryptology ePrint Archive, Paper 2025/535, 2025.

[94] Jiaheng Zhang, Tiancheng Xie, Yupeng Zhang, and Dawn Xiaodong Song. Transparent polynomial delegation and its applications to zero knowledge proof. *2020 IEEE Symposium on Security and Privacy (SP)*, pages 859–876, 2020.

[95] Yupeng Zhang, Daniel Genkin, Jonathan Katz, Dimitrios Papadopoulos, and Charalampos Papamanthou. vSQL: Verifying Arbitrary SQL Queries over Dynamic Outsourced Databases . In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 863–880, Los Alamitos, CA, USA, May 2017. IEEE Computer Society.

[96] ZKPassport. Zkpassport. https://github.com/zkpassport, 2025. Accessed: 2025-08-08.

Appendix A. UltraGroth Security Proof

First, we give the definitions of properties that UltraGroth must satisfy in order to be a secure zk-SNARK protocol.

Definition A.1. *zk-SNARK protocol* $(\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ *may have the following properties:*

1) *Perfect Completeness.* Π *is complete if for any efficient relation* $\mathcal{R}$ *the valid proof* π *is always accepted by the verifier:*

$$Pr\left[\mathsf{Verify}(\mathsf{vp}, \mathbb{x}, \pi) = 1 \;\middle|\; \begin{array}{c}(\mathsf{pp}, \mathsf{vp}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}) \\ \pi \leftarrow \mathsf{Prove}(\mathsf{pp}, \mathbb{x}, \mathbb{w}) \\ (\mathbb{x}, \mathbb{w}) \in \mathcal{R}\end{array}\right] = 1$$

2) *Perfect Zero-Knowledge.* Π *provides perfect zero-knowledge if for any efficient relation* $\mathcal{R}$*, there exists PPT simulator* Sim *such that for any* $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$*, the distribution of a real proof is identical to the distribution of a simulated proof. Formally, after a trusted setup* $(\mathsf{pp}, \mathsf{vp}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R})$*, the following two distributions are identical:*

$$[\pi \leftarrow \mathsf{Prove}(\mathsf{pp}, \mathbb{x}, \mathbb{w}) : \pi] \approx \mathsf{Sim}(1^\lambda, \mathsf{vp}, \mathbb{x})$$

3) *Computational Knowledge Soundness.* Π *is a computational proof of knowledge if for any efficient relation* $\mathcal{R}$ *and for any PPT malicious prover* $\mathcal{P}^*$*, there exists a PPT extractor* $\mathcal{E}$ *which can "extract" the secret witness* $\mathbb{w}$ *from* $\mathcal{P}^*$*. Formally, if a prover* $\mathcal{P}^*$ *can produce a valid proof* π *for a statement* $\mathbb{x}$ *with non-negligible probability, then the extractor (given oracle access to* $\mathcal{P}^*$*) can produce a valid witness* $\mathbb{w}^*$ *for that statement:*

$$\Pr\left[\mathsf{Verify}(\mathsf{vp}, \mathbb{x}, \pi) = 1 \;\middle|\; \begin{array}{c}(\mathsf{pp}, \mathsf{vp}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}); \\ \pi \leftarrow \mathcal{P}^*(\mathsf{pp}, \mathbb{x}); \\ \mathbb{w}^* \leftarrow \mathcal{E}^{\mathcal{P}^*}(\mathsf{pp}, \mathbb{x})\end{array}\right] \geq 1 - \mathsf{negl}(\lambda)$$

Theorem A.1. *UltraGroth protocol is a secure zk-SNARK protocol, i.e., it satisfies perfect completeness, perfect zero-knowledge, and computational knowledge soundness.*

Proof. *Completeness* is easy to show. Recall that the verifier has to check whether

$$e(\pi_A, \pi_B) = e(g_1^\alpha, g_2^\beta) \cdot e(\pi_{\mathrm{IC}}, g_2^\gamma) \cdot \prod_{i \in [d+1]} e(\pi_C^{\langle i \rangle}, g_2^{\delta_i})$$

Consider the expression $m_C := \prod_{i \in [d+1]} e(\pi_C^{\langle i \rangle}, g_2^{\delta_i})$. Let $g_T := e(g_1, g_2)$, then:

$$m_C = \prod_{i \in [d+1]} e(\pi_C^{\langle i \rangle}, g_2^{\delta_i}) = g_T^{\sum_{i \in [d]} \delta_i c_i(\tau) + \delta_d c_d(\tau)} =: g_T^{\zeta_C}$$

Now we expand the exponent:

$$\begin{aligned}\zeta_C &= \sum_{i \in [d]} \delta_i \left(\sum_{j \in \mathcal{I}_W^{\langle i \rangle}} z_j \frac{\zeta_j(\tau)}{\delta_i} + r_i \delta_d \right) + \delta_d c_d(\tau) \\ &= \sum_{i \in [d]} \sum_{j \in \mathcal{I}_W^{\langle i \rangle}} z_j \zeta_j(\tau) + \delta_d \sum_{i \in [d]} \delta_i r_i + \delta_d c_d(\tau)\end{aligned}$$

Now, notice that $\sum_{i\in[d]}\sum_{j\in\mathcal{I}_W^{\langle i\rangle}}$ is essentially summing over index array $\bigcup_{i\in[d]}\mathcal{I}_W^{\langle i\rangle}$, which in turn is the set $\mathcal{I}_W\setminus\mathcal{I}_W^{\langle d\rangle}$. Additionally, we expand $c_d(\tau)$:

$$\zeta_C = \sum_{j\in\mathcal{I}_W\setminus\mathcal{I}_W^{\langle d\rangle}} z_j\zeta_j(\tau) + \sum_{i\in\mathcal{I}_W^{\langle d\rangle}} z_i\zeta_i(\tau) + h(\tau)t(\tau) + \delta_d a(\tau)s + \delta_d b(\tau)r - rs\delta_d^2$$

Note that $\sum_{j\in\mathcal{I}_W\setminus\mathcal{I}_W^{\langle d\rangle}} + \sum_{j\in\mathcal{I}_W^{\langle d\rangle}} = \sum_{j\in\mathcal{I}_W}$. Hence, we further simplify the expression:

$$m_C = g_T^{\sum_{j\in\mathcal{I}_W} z_j\zeta_j(\tau)+h(\tau)t(\tau)+\delta_d a(\tau)s+\delta_d b(\tau)r-rs\delta_d^2}$$

But notice that this expression is nothing but

$$m_C = e\left(g_1^{\widetilde{c}_d(\tau)}, g_2^{\delta_d}\right),$$

$$\widetilde{c}_d(X) = \frac{1}{\delta_d}\left(\sum_{j\in\mathcal{I}_W} z_j\zeta_j(X) + h(X)t(X)\right) + a(X)s + b(X)r - rs\delta_d$$

This is exactly the expression that we had in the original Groth16's verifier equation, but instead of a single δ, only the last coefficient δ_d is used. Hence, the correctness of the UltraGroth verifier is guaranteed by the correctness of the original Groth16 verifier.

Now we show <u>*perfect zero-knowledge*</u>. Note that the pairing check verifies whether $a(\tau)b(\tau) = \alpha\beta + \mathsf{IC}(\tau)\gamma + \sum_{i\in[d+1]}\delta_i c_i(\tau)$. For that reason, we propose the following simulator Sim that outputs a valid proof $\boldsymbol{\pi}$ that is indistinguishable from the real proof.

<u>**Simulator** $\mathsf{Sim}(1^\lambda, \mathsf{vp}, \mathbb{x}) \to \boldsymbol{\pi}$:</u>

1) *Sample uniformly random $a', b', \{c_i'\}_{i\in[d]} \xleftarrow{R} \mathbb{F}$, corresponding to the exponents of corresponding proof elements.*
2) *Output the proof $\boldsymbol{\pi} = (g_1^{a'}, g_2^{b'}, \{g_1^{c_i'}\}_{i\in[d+1]})$ where:*

$$c_d' = \frac{a'b' - \alpha\beta - \sum_{j\in\mathcal{I}_X} z_j\zeta_j(\tau) - \sum_{i\in[d]} c_i'\delta_i}{\delta_d}$$

Let us show why it works. The proof correctness follows directly from the verification equation. Note that first $d+2$ components of the proof are uniformly distributed groups elements (since the corresponding powers are uniformly distributed) while the last term c_d' is computed uniquely from other coefficients; thus it is also uniformly random. Hence, the UltraGroth protocol is perfectly zero-knowledge.

Finally, we show <u>*computational knowledge soundness*</u> against adversaries using affine strategies and assuming soundness of Fiat-Shamir heuristic. In order to do that, we will need the following lemma:

Lemma. Any (passing) adversary's output $\pi_C^{\langle i\rangle}$ using affine strategy for $i\in[d]$ commits to *some* valid $\mathbb{w}_i$'s (i.e., does not involve z_j for $j\notin\mathcal{I}_W^{\langle i\rangle}$).

Our plan to establish this lemma is as follows: we will perform some transformations with the linear combination output by the prover, and inspect the terms involved carefully. We generalize the results closely following the original Groth16 paper [37].

Recall that in the affine strategy, the adversary computes the proof by applying the linear transformation using matrix Π on the common reference string: $\boldsymbol{\pi} = \Pi\boldsymbol{\sigma} = (a(\tau), b(\tau), \{c_i(\tau)\}_{i\in[d+1]}) \in \mathbb{F}^{d+3}$. Here,

$$\boldsymbol{\sigma} = \Bigg(\alpha, \beta, \gamma, \{\delta_i\}_{i\in[d+1]}, \{\tau^i\}_{i\in[n]}, \left\{\frac{\zeta_i(\tau)}{\gamma}\right\}_{i\in\mathcal{I}_X}, \left\{\frac{\zeta_j(\tau)}{\delta_i}\right\}_{i,j\in[d+1]\times\mathcal{I}_W^{\langle i\rangle}}, \left\{\frac{\tau^i t(\tau)}{\delta_d}\right\}_{i\in\mathcal{I}_W^{\langle d\rangle}}\Bigg)$$

Prover's strategy for the first part a is following:

$$a = a_\alpha\alpha + a_\beta\beta + a_\gamma\gamma + \sum_{i\in[d+1]} a_\delta^{\langle i\rangle}\delta_i + a(\tau) + \sum_{i\in\mathcal{I}_X} a_i\frac{\zeta_i(\tau)}{\gamma} + \sum_{i\in[d+1]}\sum_{j\in\mathcal{I}_W^{\langle d\rangle}} a_j\frac{\zeta_j(\tau)}{\delta_i} + a_h(\tau)\frac{t(\tau)}{\delta_d},$$

where field elements $a_\alpha, a_\beta, a_\gamma, \{a_\delta^{\langle i\rangle}\}_{i\in[d+1]}, \{a_i\}_{i\in\mathcal{I}_W}$ and polynomials $a(\tau)$ and $a_h(\tau)$ are known and correspond to the first row of the matrix Π. The same structure, albeit with different coefficients, holds for b and $\{c_i\}_{i\in[d+1]}$.

Now we view the verification equation as an equality of multivariate Laurent polynomials. By the Schwartz-Zippel lemma, the prover has a negligible probability for making the verification equation hold for randomly selected indeterminates $\alpha, \beta, \gamma, \{\delta_i\}_{i\in[d+1]}$ and τ. Now recall that the check is the following:

$$ab = \alpha\beta + \mathsf{IC}(\tau)\gamma + \sum_{i\in[d+1]}\delta_i c_i.$$

Now we are going to derive what the coefficients of the corresponding polynomials a, b, and $\{c_i\}_{i\in[d+1]}$ are. The terms with indeterminate α^2 are $a_\alpha b_\alpha\alpha^2 = 0$ and so $a_\alpha = 0$ or $b_\alpha = 0$. Without loss of generality, assume $b_\alpha = 0$. Terms with $\alpha\beta$ gives us $a_\alpha b_\beta + a_\beta b_\alpha = a_\alpha b_\beta = 1$. Again, without loss of generality, assume $a_\alpha = 1$ and $b_\beta = 1$. Finally, the indeterminate β^2 gives $a_\beta b_\beta = a_\beta = 0$. Therefore, the expressions for a and b are now simplified down to:

$$a = \alpha + a_\gamma\gamma + \sum_{i\in[d+1]} a_\delta^{\langle i\rangle}\delta_i + \ldots,$$

$$b = \beta + b_\gamma\gamma + \sum_{i\in[d+1]} b_\delta^{\langle i\rangle}\delta_i + \ldots$$

Now consider the terms involving $1/\delta_d^2$. We have:

$$\left(\sum_{j\in\mathcal{I}_W^{\langle d\rangle}} a_j\zeta_j(\tau) + a_h(\tau)t(\tau)\right) \times \left(\sum_{j\in\mathcal{I}_W^{\langle d\rangle}} b_j\zeta_j(\tau) + b_h(\tau)t(\tau)\right) = 0$$

Without loss of generality, assume $\sum_{j\in\mathcal{I}_W^{\langle d\rangle}} a_j\zeta_j(\tau) + a_h(\tau)t(\tau) = 0$. However, looking at term $\frac{\alpha}{\delta_d}\sum_{j\in\mathcal{I}_W^{\langle d\rangle}} b_j\zeta_j(\tau) + a_h(\tau)t(\tau) = 0$ shows that in fact $\sum_{j\in\mathcal{I}_W^{\langle d\rangle}} b_j\zeta_j(\tau) + a_h(\tau)t(\tau) = 0$ as well.

Now, consider the terms with $1/\delta_i^2$ for $i\neq d$. We have:

$$\sum_{j\in\mathcal{I}_W^{\langle i\rangle}} a_j\zeta_j(\tau)\cdot\sum_{j\in\mathcal{I}_W^{\langle i\rangle}} b_j\zeta_j(\tau) = 0$$

Without loss of generality assume $\sum_{j\in\mathcal{I}_W^{\langle d\rangle}} a_j\zeta_j(\tau) = 0$. Again, looking at term $\frac{\alpha}{\delta_i}\sum_{j\in\mathcal{I}_W^{\langle i\rangle}} b_j\zeta_j(\tau) = 0$ shows that in fact $\sum_{j\in\mathcal{I}_W^{\langle i\rangle}} b_j\zeta_j(\tau) = 0$ as well.

Now terms involving $1/\gamma^2$ give us

$$\sum_{j\in\mathcal{I}_X} a_j\zeta_j(\tau)\cdot\sum_{j\in\mathcal{I}_X} b_j\zeta_j(\tau) = 0$$

For the very same reason, we derive that both sums are zero. Finally, the terms $a_\gamma\beta\gamma = 0$ and $b_\gamma\alpha\gamma = 0$ show that $a_\gamma = 0$ and $b_\gamma = 0$. Therefore, we have

$$a = \alpha + a(\tau) + \sum_{i\in[d+1]} a_\delta^{\langle i\rangle}\delta_i,\ b = \beta + b(\tau) + \sum_{i\in[d+1]} b_\delta^{\langle i\rangle}\delta_i$$

Now, let us write the product ab explicitly:

$$\begin{aligned} ab &= \cancel{\alpha\beta} + \alpha b(\tau) + \alpha\sum_{i\in[d+1]} b_\delta^{\langle i\rangle}\delta_i + \beta a(\tau) \\ &+ a(\tau)b(\tau) + a(\tau)\sum_{i\in[d+1]} b_\delta^{\langle i\rangle}\delta_i + \sum_{i\in[d+1]} a_\delta^{\langle i\rangle}\delta_i\beta \\ &+ \sum_{i\in[d+1]} a_\delta^{\langle i\rangle}\delta_i b(\tau) + \sum_{i\in[d+1]} a_\delta^{\langle i\rangle}\delta_i\sum_{j\in[d+1]} b_\delta^{\langle j\rangle}\delta_j \\ &= \cancel{\alpha\beta} + \sum_{j\in\mathcal{I}_X} z_j(\beta\ell_i(\tau) + \alpha r_i(\tau) + o_i(\tau)) + \sum_{i\in[d+1]}\delta_i c_i \end{aligned}$$

Consider the terms that involve α (but not $\alpha\delta_i$ and such!) and β. We have:

$$\begin{aligned} b(\tau) &= \sum_{j\in\mathcal{I}_X} z_j r_i(\tau) + \sum_{j\in\mathcal{I}_W} c_j r_j(\tau), \\ a(\tau) &= \sum_{j\in\mathcal{I}_X} z_j\ell_i(\tau) + \sum_{j\in\mathcal{I}_W} c_j\ell_j(\tau) \end{aligned}$$

Thus, define $z_j := c_j$ for $j\in\mathcal{I}_W$. This way, we have

$$a(\tau) = \sum_{j\in[n]} z_j\ell_j(\tau),\quad b(\tau) = \sum_{j\in[n]} z_j r_j(\tau)$$

Finally, looking at the terms involving powers of τ we have:

$$\sum_{j\in[n]} z_j\ell_j(\tau)\cdot\sum_{j\in[n]} z_j r_j(\tau) = \sum_{j\in[n]} z_j o_j(\tau) + c_h(\tau)t(\tau)$$

This shows that $\{z_j\}_{j\in\mathcal{I}_W} \equiv \{c_j\}_{j\in\mathcal{I}_W}$ is a witness for the statement $\mathbb{x} = \{z_j\}_{j\in\mathcal{I}_X}$, and, in particular, that $\pi_C^{\langle i\rangle}$ is a binding commitment to $\mathbb{w}_i$ part of the witness.

This concludes the proof, as UltraGroth protocol can than be treated as Fiat-Shamir transformation of the original protocol (with respect to slightly unusual but valid compression function, that hashes the binding commitment to $\mathbb{w}_i$). According to the lemma above, any affine adversary that manages to satisfy the final check is forced to output a valid transcript for such a protocol.

Appendix B. UltraGroth Efficiency

In this appendix, we analyze the efficiency of the UltraGroth protocol compared to the original Groth16 protocol. First, consider the following theorem that gives the optimal value of chunk size w.

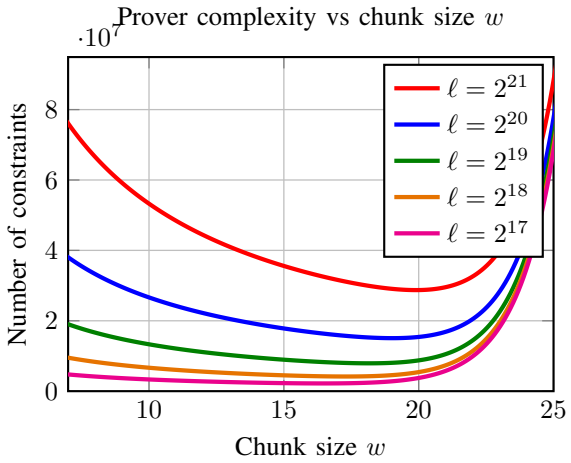

Figure 8: The circuit size depending on the chunk size w. The different colors represent different values of ℓ.

Theorem B.1. *The optimal chunk size $\widehat{w}$ is the solution to equation: $2^w w^2 = n/\ln 2$ (for $n = \ell\lambda$).*

Proof. We first find the derivative of the cost function $c(w) = 2^w + \frac{n}{w}$ and set it to 0 (we depict the cost function $c(w)$ in Figure 8). The derivative is $c'(w) = 2^w\ln 2 - \frac{n}{w^2}$. Equating this to 0 yields the equation above. To show it is a minimum, we note that the second derivative is positive: $c''(w) = 2^w\ln^2 2 + \frac{2n}{w^3}$, thus $c''(\widehat{w}) > 0$.

Now, we analyze the assymptotics of the protocol.

Theorem B.2. *Suppose the circuit consists mainly of r range checks, thus requiring $n \approx \lambda r$ constraints in the original Groth16 protocol. With the optimal chunk size, the UltraGroth protocol requires only $\mathcal{O}(n/\log n)$ constraints to be satisfied.*

Proof. The optimal chunk size must satisfy the equation $2^w w^2 = n/\ln 2$ according to Theorem B.1. Therefore, we have $n = \ln 2\cdot 2^w w^2$ so the total cost in terms of w solely is $c(w) = 2^w(1 + w\ln 2) \sim w\cdot 2^w$.

We do the same for $g(n) = n/\ln n$. We have $g(w) = \ln 2\cdot 2^w w^2/\ln(\ln 2\cdot 2^w w^2) \sim 2^w w^2/(w + \ln w) \sim w\cdot 2^w$. As we see, $c \sim g$ and thus the UltraGroth protocol complexity is assymptotically $\mathcal{O}(n/\log n)$.

Appendix C. R1CS-Friendly Architectures

C.1. Activation Functions

While we greatly benefit from linear layers, we cannot avoid computing the non-linear functions. Unfortunately, many activations such as *sigmoid* (corresponding to $\sigma(x) = \frac{1}{1+e^{-x}}$) or *tanh* (corresponding to $\tanh(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$) contain exponentials and thus become highly problematic when it comes to the R1CS implementation, comprising the finite field arithmetic. While there are approaches to handle such activations using polynomial approximations [51] or even specialized table lookup-based protocols [41], we decide to limit ourselves to the list of *ReLU-based* activations, which can be implemented precisely and effectively without loss of accuracy. We

show that many activations such as *swish*, *sigmoid*, and even *GELU* can be substituted with the ReLU-based activations.

It is worth noting that the loss of accuracy is not the primary issue with polynomial-based activations, which seem promising based on Π-nets [20], showing that even the Face Recognition neural network can be implemented with the verification performance of over 99.8% using polynomial activations. Their main problem is that polynomial approximations cause the significant raise in precision: although computing $Q(\widehat{\mathbf{x}})$ for d-degree polynomial $Q \in \mathbb{F}[x]$ requires only $d \ll b$ constraints per value, such computation increases the precision of $\widehat{\mathbf{x}}$ by ρd bits. Consequently, even moderately deep networks would require cutting precision of a large number of values, each costing roughly b constraints. This way, the benefit of d constraints per value would be lost in the overhead of precision cuts.

This motivates us to use the ReLU-based activations where we can not only compute the activation function with $b+1$ constraints, but also cut the precision for free, which was discussed in § 4.

Other available non-linearities. One obvious activation function that is easily implementable using our approach is the *LeakyReLU* activation, which is defined as $\max\{\alpha x, x\}$ for some fixed $\alpha \in [0, 1]$. Note that if α happens to be the power of two: $\alpha = 2^{-E}$ for $E > 0$, we can compute this activation in $b+1$ constraints, just as ReLU. The trick here is that when we perform the shift by $\delta\rho$ bits, we can perform an additional shift by E bits, which is simply equivalent to bit-shift by $\delta\rho + E$ bits.

Is there any other functions we can effectively implement? Inspired by the *MobileNetV3* [44] paper, we can also switch *sigmoid* and *swish* activations by what the paper calls *hard* activations, which are defined as follows:

$$\text{hard-}\sigma(x) \triangleq \frac{1}{6}\text{ReLU6}(x-3),$$
$$\text{hard-swish}(x) \triangleq x \cdot \text{hard-}\sigma(x),$$

where $\text{ReLU6}(x) \triangleq \min(\text{ReLU}(x), 6)$. With some care, both these functions can be implemented in approximately b constraints. Finally, we can also implement the *GELU* activation function using the widely used approximation $x\sigma(1.703x)$. Since sigmoid function is not enabled, we switch it to the hard alternative: $\text{GELU}(x) \approx x \cdot \text{hard-}\sigma(1.703x)$

C.2. Encoder-Decoder Dense Layer

Now, given the fact that the neural network has L non-linear activation calls, the approximate number of constraints for the whole network is roughly Lb in Groth16. Unfortunately, this number is still high for the modern neural networks since the number of non-linearities is typically in the order of tens of thousands. We solve this issue by (a) introducing the *encoder-decoder layers*, allowing to reduce L while maintaining the expressiveness of the neural network, and (b) use the UltraGroth protocol, allowing reducing the number of constraints down to Lb/w for $w \approx 20$. While point (b) was discussed in § 5, we focus on point (a) now.

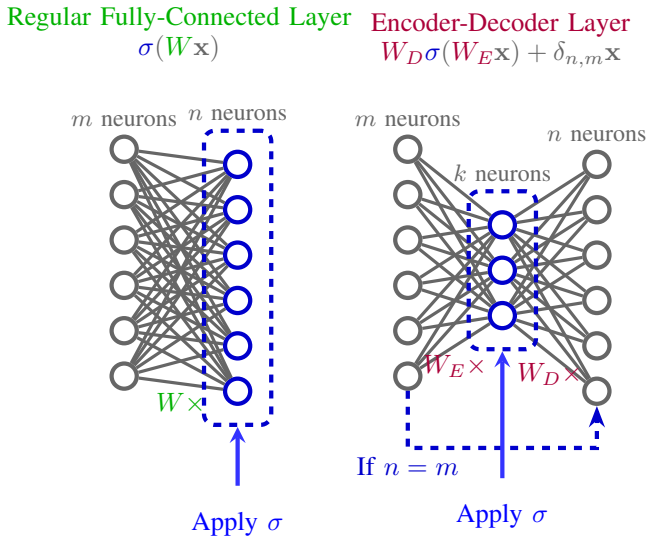

Figure 9: The visual difference between the regular fully-connected layer and the encoder-decoder layer. The encoder-decoder layer consists of two linear layers (encoder W_E and decoder W_D) and a non-linear activation function σ applied to the latent space. The regular fully-connected layer applies the activation function to the output of the layer obtained by the linear projection W.

To restate the issue, assume at some point the output of the layer is $\mathbf{x} \in \mathbb{F}^n$ and we apply[5] the fully-connected layer $\mathsf{FCLayer}(\mathbf{x}; W) = \sigma(W\mathbf{x})$ with $W \in \mathbb{F}^{m\times n}$. As we have seen, this costs approximately mb to verify. Can we reduce this cost? To solve this issue, *we (a) inject an additional layer with k hidden neurons, (b) apply the non-linearity σ at this newly added hidden layer, and (c) remove the non-linearity from the output layer.* This way, the encoder-decoder layer would first project the input $\mathbf{x} \in \mathbb{R}^n$ to the latent space $\mathbf{z} \in \mathbb{R}^k$ using the linear (encoder) layer $\mathbf{z} = W_E\mathbf{x}$, then apply the non-linearity σ to the latent space $\mathbf{z}$ and finally project the result back to the output space $\mathbf{y} \in \mathbb{R}^m$ using the linear (decoder) layer $\mathbf{y} = W_D\sigma(\mathbf{z})$. All in all, we get the forward pass $\mathbf{y} = W_D\sigma(W_E\mathbf{x})$. Finally, following the original ResNet paper [42], we add the *residual connection* between the input and output of the layer in case $n = m$.

Definition C.1. *The **Encoder-Decoder Layer** with n inputs and m outputs consists of two linear layers (encoder and decoder) and a non-linear activation function σ applied to the latent space of the **hidden size** k. We define it as:*

$$\mathsf{EDLayer}(\mathbf{x}; W_D, W_E) = \begin{cases} W_D\sigma(W_E\mathbf{x}) + \mathbf{x}, & \text{if } n = m \\ W_D\sigma(W_E\mathbf{x}), & \text{if } n \neq m \end{cases}$$

*where $W_E \in \mathbb{R}^{k\times n}$ is the encoder matrix and $W_D \in \mathbb{R}^{m\times k}$ is the decoder matrix. We call the ratio $\gamma := \frac{k}{m} < 1$ the **squeezing factor** of the ED Layer. See depicted flow in Figure 9.*

Of course, this way some accuracy is lost, but our implementations of liveness and face recognition neural

5. For notation simplicity, we do not write bias terms in the MLP equations further on by assuming that the input activations are passed with the "1" appended.

networks indicate that the accuracy loss can be insignificant for the proper architecture. Such architecture is mainly beneficial because of the following fact.

Proposition C.1. *The encoder-decoder layer can be implemented in R1CS with roughly kb constraints instead of mb, thus giving the speedup of $1/\gamma$ in contrast to the regular fully-connected layer* $\mathsf{FCLayer}(\mathbf{x}; W)$.

C.3. Encoder-Decoder Convolutional Layers

While we described how to optimize fully-connected layers, we still need the anologous construction for the convolutional layers, which operate over images. Recall that in the convolutional networks, we use mappings from one image tensor $\boldsymbol{X} \in \mathbb{R}^{W \times H \times C}$ (stack of C images of size $W \times H$) to another image tensor $\boldsymbol{Y} \in \mathbb{R}^{W' \times H' \times C'}$. This is typically done using the C' kernels of size $f \times f$, which we denote by $\{K_c\}_{c \in [C']} \subseteq \mathbb{R}^{f \times f}$ with stride s which convolves the input image as follows:

$$Y_{i,j,c} = \sigma\left(\sum_{k \in [C]} \sum_{u \in [f]} \sum_{v \in [f]} K_{u,v,c} X_{i \cdot s+u, j \cdot s+v, k} + \beta_{i,j,c}\right)$$

where the output volume $\boldsymbol{Y}$ is of size $\frac{W}{s} \times \frac{H}{s} \times C'$. Now, the primary issue with this version of convolution is that it requires computing roughly $H'W'C'$ non-linear activations (essentially, the whole output volume). To mitigate this issue, we need to come up with the way to reduce the number of non-linear activations.

C.3.1. Squeeze-and-Excitation Block. One of such examples in the existing literature is the *Squeeze-and-Excitation* (SE) block [46]. The original network proceeds as follows: given the input volume $\boldsymbol{X} \in \mathbb{R}^{W \times H \times C}$, we apply the *global average pooling* operation to get the vector $\mathbf{z} \in \mathbb{R}^C$, where each z_c is the average of the c-th channel of the input volume:

$$z_c = \frac{1}{W \cdot H} \sum_{i=1}^{W} \sum_{j=1}^{H} X_{i,j,c}, \quad c \in [C]$$

Next, we apply the gating mechanism to the vector $\mathbf{z}$ to set $\mathbf{s} \in \mathbb{R}^C$, which essentially is the encoder-decoder layer, but with the non-linearity applied at the end: $\mathbf{s} = g(W_D \sigma(W_E \mathbf{z}))$. Here, the number of neurons in the hidden layer is $\frac{C}{r}$ for some constant r. Finally, we multiply each channel of the input volume $\boldsymbol{X}$ by the corresponding s_c to get the output volume: $\boldsymbol{Y}[:,:,c] \leftarrow s_c \cdot \boldsymbol{X}[:,:,c]$. The whole computation flow is illustrated in Figure 10. We give the following proposition to get the number of constraints.

Proposition C.2. *Given that both σ and g cost b constraints to verify, the total cost of the squeeze-and-excitation block is approximately $\left(1 + \frac{1}{r}\right) Cb + HWC$.*

Reasoning. Since all linearities are free, we only need to (a) check the activation over the hidden layer, (b) check the activation over the output layer and (c) check the multiplication of the input volume with the output of the SE block. The first step costs $\frac{C}{r} \cdot b$ constraints, the second step costs Cb constraints and the third step costs HWC constraints. The total cost is therefore $\left(1 + \frac{1}{r}\right) Cb + HWC$.

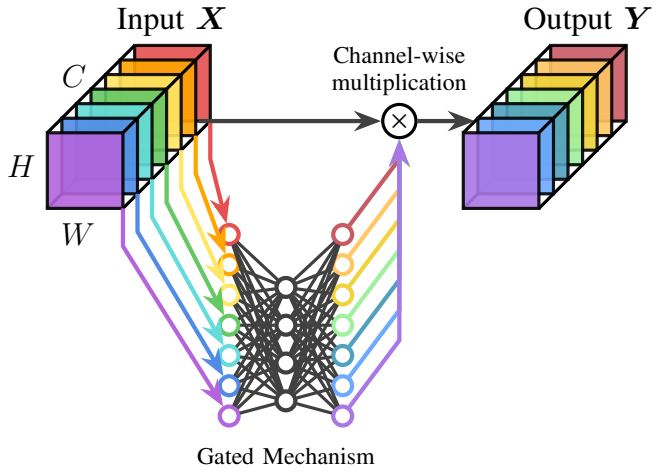

Figure 10: The Squeeze-and-Excitation (SE) block from [46] — one of R1CS-friendly layers

Heavy SE Block. One modification of such block is the following: instead of applying the global average pooling along the whole volume, we can effectively split it into blocks (say, forming the grid of size $P_W \times P_H$). This way, we increase the number of constraints by the factor of $P_W P_H$, but the neural network can learn more complex representations.

C.3.2. Encoder-Decoder Convolutional Layer. The issue with the Squeeze-and-Excitation Block is that it does not change the shape of the volume. For that reason, we give the more interesting construction, which is the *encoder-decoder convolutional layer*. We combine certain ideas from the Vision Transformer [25] and the previously mentioned squeeze-and-excitation block [46].

First, we split the input volume into blocks of size $P_W \times P_H$. For simplicity, assume $P_W = P_H = P$ and naturally assume $P \mid W, H$. Assuming the input volume is $\boldsymbol{X} \in \mathbb{R}^{W \times H \times C}$, we get P^2 blocks $\boldsymbol{X}_{i,j} \in \mathbb{R}^{\frac{W}{P} \times \frac{H}{P} \times C}$ for $i, j \in [P]$. Then, we flatten each block into the vector of size $\frac{WHC}{P^2}$. We connect the flattened blocks to the encoder-decoder layer, each with the K hidden units. Finally, assume we want to get W', H' and C' as the output volume. This way, we connect the latent layer (of size K) to the output layer of size $\frac{W'H'C'}{P^2}$. Finally, we reshape the output volume into the volume of size $\frac{W'}{P} \times \frac{H'}{P} \times C'$ and combine the blocks together to get the final output $\boldsymbol{Y} \in \mathbb{R}^{W' \times H' \times C'}$.

Now, let us analyze the cost of such convolution.

Proposition C.3. *Given that the non-linearity in the encoder-decoder layer costs b constraints, the total cost of the encoder-decoder convolutional layer is approximately $P^2 Kb$.*

Reasoning. The only non-linear operation in the encoder-decoder layer is the activation function. Since each processing of encoder-decoder costs Kb constraints and we need to process P^2 blocks, the total cost is $P^2 Kb$.

Note that while the regular convolutional layer costs $W'H'C'b$ constraints, we reduce it to $P^2 Kb$ constraints.

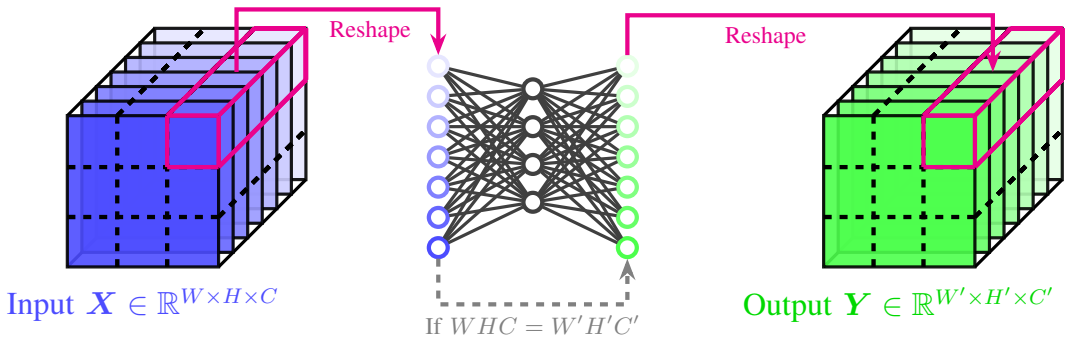

Figure 11: The encoder-decoder convolutional layer. The input volume is split into blocks of size $P \times P \times C$ and the encoder-decoder layer is applied to each block. The output is obtained by combining the outputs of processed blocks with the proper reshaping.

Appendix D. Quantization Scheme Proof

In this appendix, we show that the Bionetta quantization scheme introduced in § 4 is sound and does not introduce significant errors in the inference process.

Theorem D.1. *Let $\widehat{x} := Q_\rho(x)$ and $\widehat{y} := Q_\rho(y)$ be the quantized values of x and y, respectively. Then, if we define circuit $\mathtt{C}_f(\widehat{x}, \widehat{y}) = \widehat{x} + \widehat{y}$ for the addition $f(x, y) = x + y$, the following relation for an error ε_ρ holds:*

$$\varepsilon_\rho := |D_\rho(\mathtt{C}_f(\widehat{x}, \widehat{y})) - f(x, y)| \leq 2 \cdot 2^{-\rho}$$

Consequently, the error ε_ρ is negligible in ρ.

Proof. For simplicity, assume $x, y > 0$ (the negative case, with more care, can be handled similarly). Since $\widehat{x} = Q_\rho(x)$ and $\widehat{y} = Q_\rho(y)$, we have $\widehat{x} = 2^\rho x + \delta_x$ for $|\delta_x| < 1$. Same holds for y: we have $\widehat{y} = 2^\rho y + \delta_y$ with $|\delta_y| < 1$. Then, it is easy to see that:

$$\begin{aligned}\mathtt{C}_f(\widehat{x}, \widehat{y}) &= \widehat{x} + \widehat{y} = (2^\rho x + \delta_x) + (2^\rho y + \delta_y) \\ &= 2^\rho (x + y) + (\delta_x + \delta_y)\end{aligned}$$

Dequantization of such expression gives:

$$D_\rho(\mathtt{C}_f(\widehat{x}, \widehat{y})) = x + y + 2^{-\rho}(\delta_x + \delta_y)$$

From which follows that $|D_\rho(\mathtt{C}_f(\widehat{x}, \widehat{y})) - f(x, y)| \leq 2^{-\rho}(|\delta_x| + |\delta_y|) \leq 2 \cdot 2^{-\rho}$.

Theorem D.2. *Let $\widehat{x} := Q_\rho(x)$ and $\widehat{y} := Q_\rho(y)$ be the quantized values of x and y, respectively. Then, if we define circuit $\mathtt{C}_f(\widehat{x}, \widehat{y}) = \widehat{x} \cdot \widehat{y}$ for the multiplication $f(x, y) = x \cdot y$ and assume $\beta := 2\max\{|x|, |y|\}$ to be relatively small, the following relation for an error ε_ρ holds:*

$$\varepsilon_\rho := |D_{2\rho}(\mathtt{C}_f(\widehat{x}, \widehat{y})) - f(x, y)| \leq 2^{-\rho}\beta + 2^{-2\rho}$$

Consequently, the error ε_ρ is negligible in ρ.

Proof. For simplicity, assume $x, y > 0$ (the negative case, with more care, can be handled similarly). Since β is assumed to be small, we assume that $\widehat{x} = [2^\rho x]$ and $\widehat{y} = 2^\rho y$ as well as $\widehat{x} \cdot \widehat{y}$ do not overflow the field. Then, it is easy to see that $\widehat{x} = 2^\rho x + \delta_x$ for $|\delta_x| < 1$. Same holds for y: we have $\widehat{y} = 2^\rho y + \delta_y$ with $|\delta_y| < 1$. Then,

$$\begin{aligned}\mathtt{C}_f(\widehat{x}, \widehat{y}) &= \widehat{x} \cdot \widehat{y} = (2^\rho x + \delta_x)(2^\rho y + \delta_y) \\ &= 2^{2\rho} xy + 2^\rho(\delta_x y + \delta_y x) + \delta_x \cdot \delta_y\end{aligned}$$

Now, we can dequantize the result with precision 2ρ:

$$D_{2\rho}(\mathtt{C}_f(\widehat{x}, \widehat{y})) = xy + 2^{-\rho}(\delta_x y + \delta_y x) + 2^{-2\rho}\delta_x \delta_y$$

Now note that $|\delta_x y + \delta_y x| \leq |x| + |y| \leq 2\max\{|x|, |y|\} = \beta$. It is also easy to see that $|2^{-2\rho}\delta_x \delta_y| < 2^{-2\rho}$. Therefore,

$$\begin{aligned}\varepsilon_\rho &= |D_{2\rho}(\mathtt{C}_f(\widehat{x}, \widehat{y})) - f(x, y)| \\ &= \left|2^{-\rho}(\delta_x y + \delta_y x) + 2^{-2\rho}\delta_x \delta_y\right| \\ &\leq 2^{-\rho}|\delta_x y + \delta_y x| + 2^{-2\rho}|\delta_x \delta_y| \\ &\leq 2^{-\rho}\beta + 2^{-2\rho},\end{aligned}$$

which concludes the proof. ■